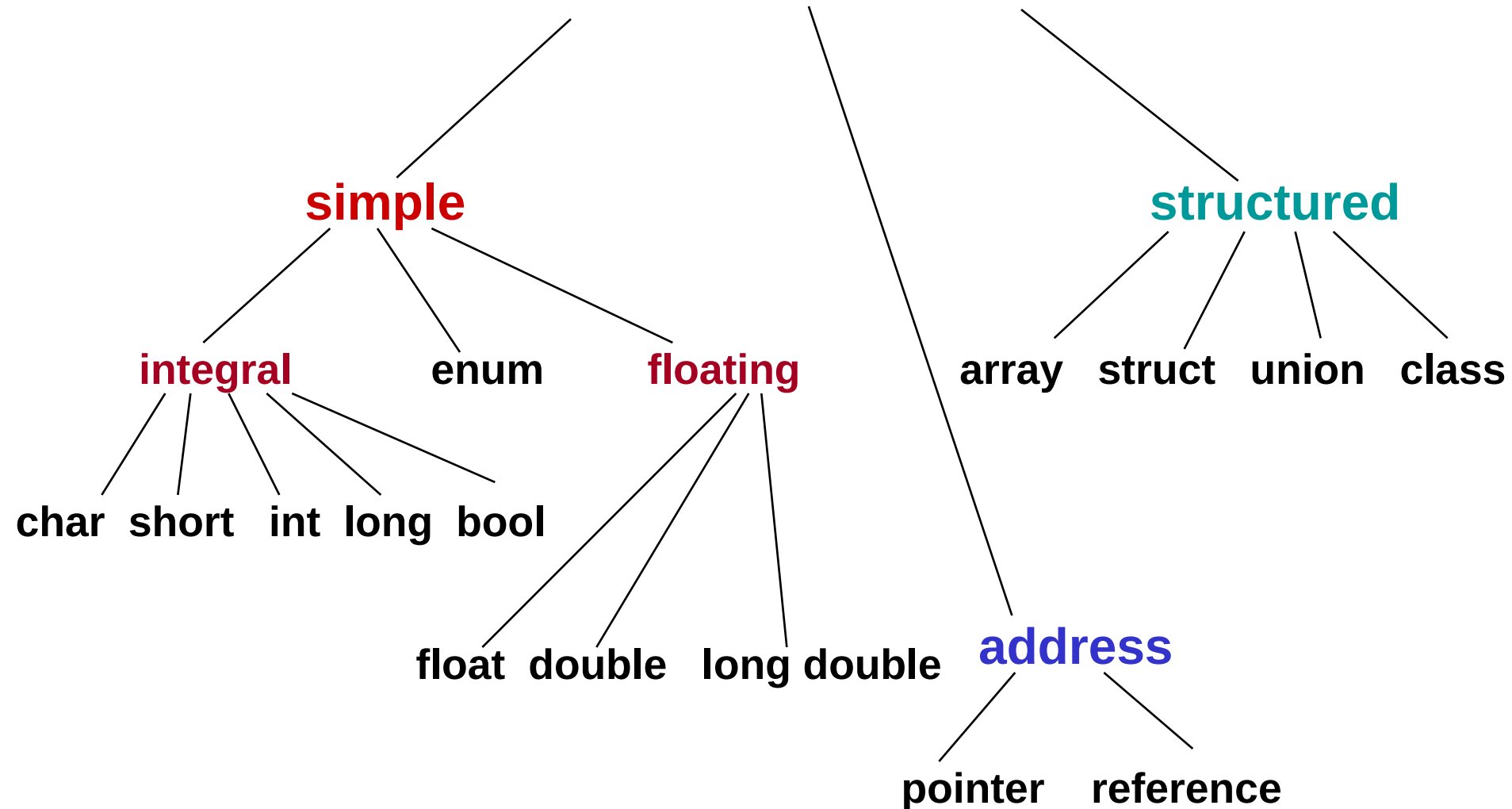


Pointers, Dynamic Data, and Reference Types

- Review on Pointers
- Reference Variables
- Dynamic Memory Allocation
 - The malloc/new
 - The free/delete
 - Dynamic Memory Allocation for Arrays

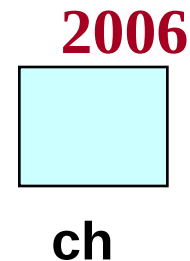
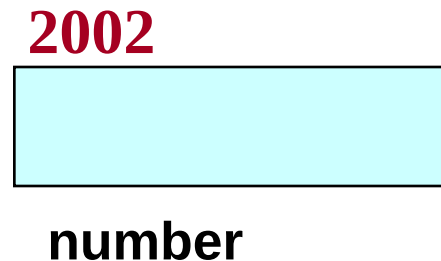
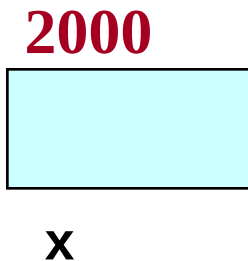
C++ Data Types



Addresses in Memory

- When a variable is declared, enough memory to hold a value of that type is allocated for it at an unused memory location. This is the address of the variable

int	x;
float	number;
char	ch;

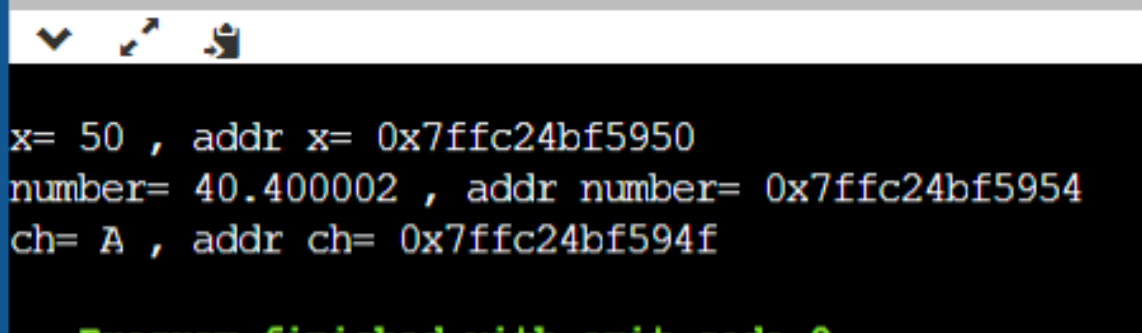


C/C++ : “&” address operator

- c/c++ supports “operator” that is able to gets a memory address of variable , the **address-of operator &**

		2000 (x)	2002 (number)	2006(ch)
int	x;			
float	number;	50	40.4	'A'
char	ch;			
cout << "Address of x is " << &x << endl;				
cout << "Address of number is " << &number << endl;				
cout << "Address of ch is " << &ch << endl;				

```
1  #include <iostream>
2  #include <stdio.h>
3  using namespace std;
4
5  int main()
6  {
7      int      x=50;
8      float    number=40.4;
9      char     ch='A';
10
11     printf("\nx= %d , addr x= %p",x,&x);
12     printf("\nnumber= %f , addr number= %p",number,&number);
13     printf("\nch= %c , addr ch= %p",ch,&ch);
14
15
16     return 0;
17 }
18
```



```
x= 50 , addr x= 0x7ffc24bf5950
number= 40.400002 , addr number= 0x7ffc24bf5954
ch= A , addr ch= 0x7ffc24bf594f
```

%p : format
display of pointer

What is a pointer variable?

- Pointer data type is a special kind of variable which are meant to store addresses only, instead of values(integer, float, double, char, etc).
- A pointer variable is a **variable whose value is the address of a location in memory.**
- To declare a pointer variable, you must specify the type of value that the pointer will point to, for example,

```
int*    ptr; // ptr will hold the address of an int
```

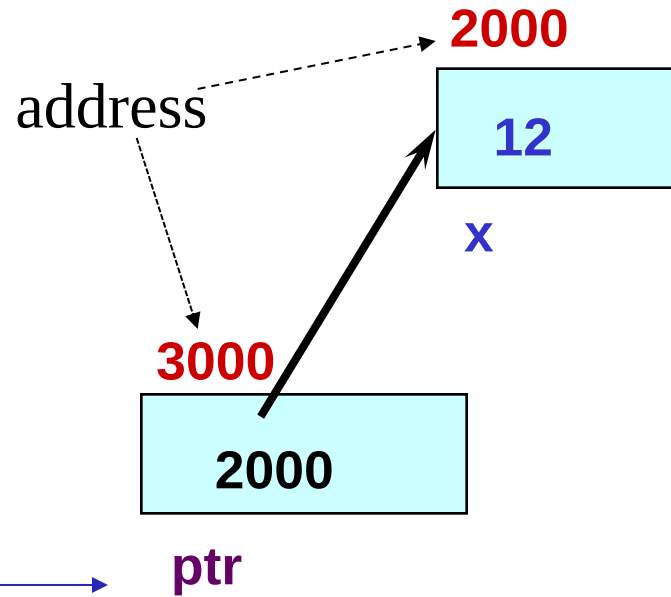
```
char*    q;    // q will hold the address of a char
```

```
şţşuşţş êñřłòýêê ê      ê xílł hólđ tğê ăđđsêşş ôğ  
          ăñ êñřłòýêê şţşuşţş
```

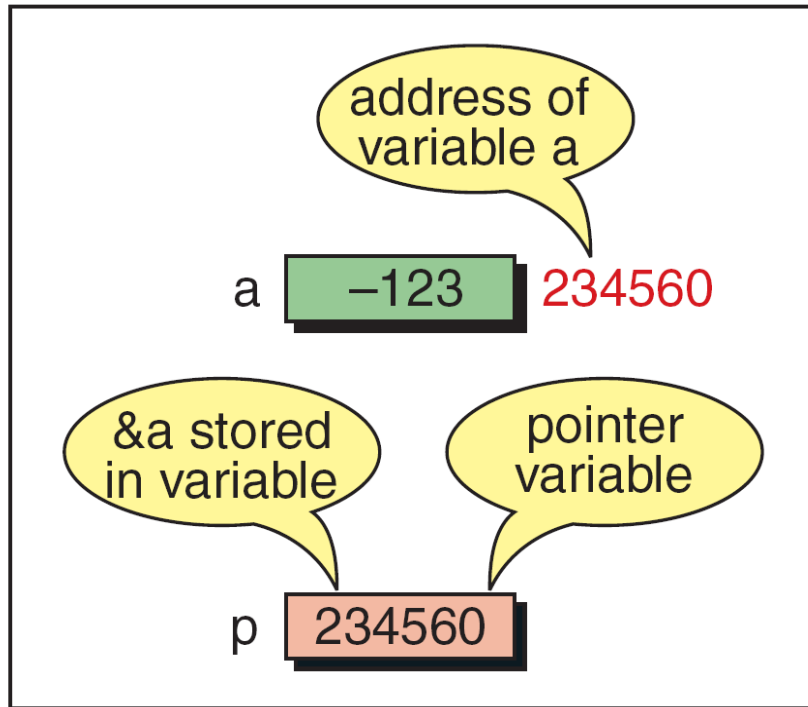
Using a Pointer Variable

```
int x;  
x = 12;
```

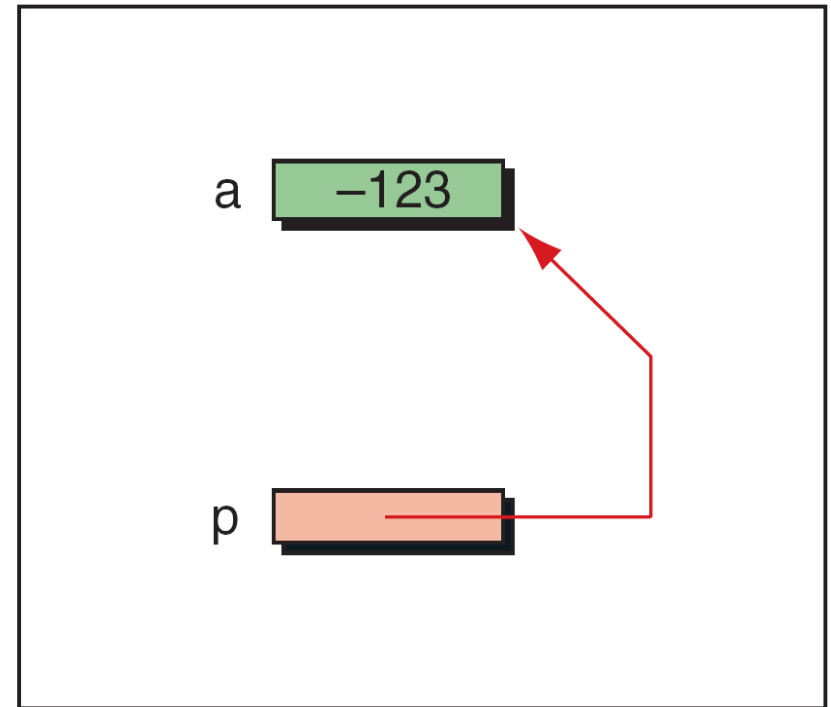
```
int* ptr;  
ptr = &x;
```



NOTE: Because **ptr** holds the address of **x**, we say that **ptr** “points to” **x**



Physical representation



Logical representation

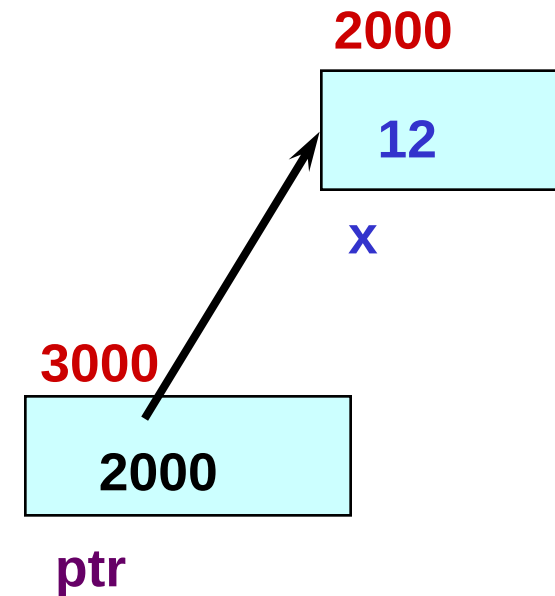
FIGURE 9-6 Pointer Variable

*****: dereference operator

```
int  x;  
x = 12;
```

```
int* ptr;  
ptr = &x;
```

```
cout << *ptr;
```

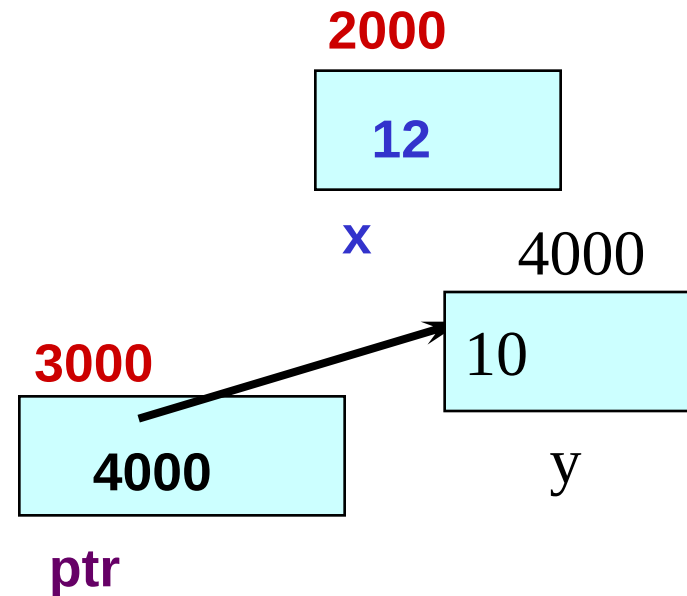


NOTE: The value pointed to by **ptr** is denoted by ***ptr**

* (dereference operator) การ access ข้อมูลที่ pointer ชี้อยู่

*: dereference operator

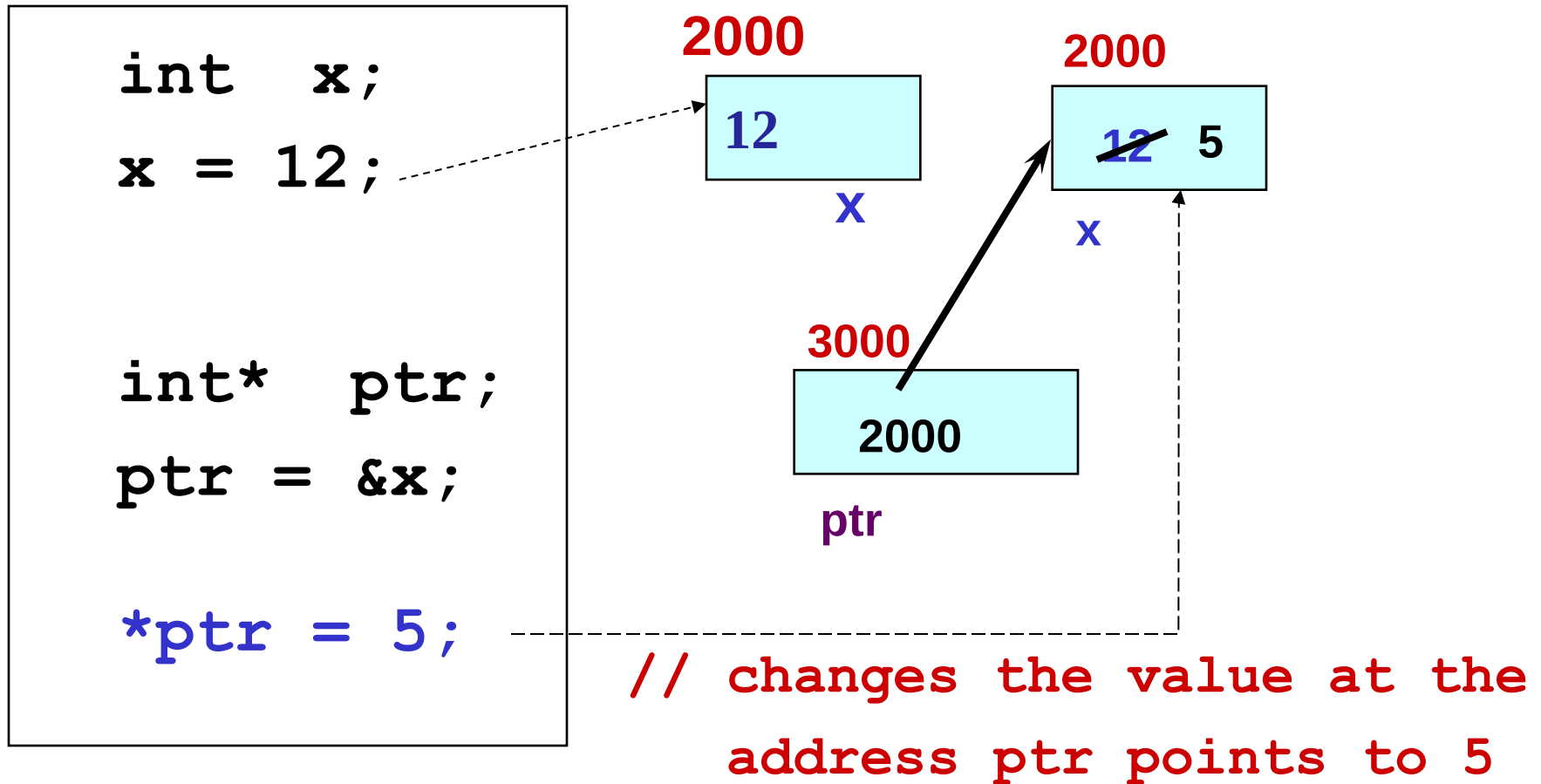
```
int x;  
x = 12;  
int y = 10;  
int* ptr;  
ptr = &x;  
cout << *ptr;  
ptr = &y;  
cout << *ptr;  
cout << &ptr;  
cout << ptr;
```



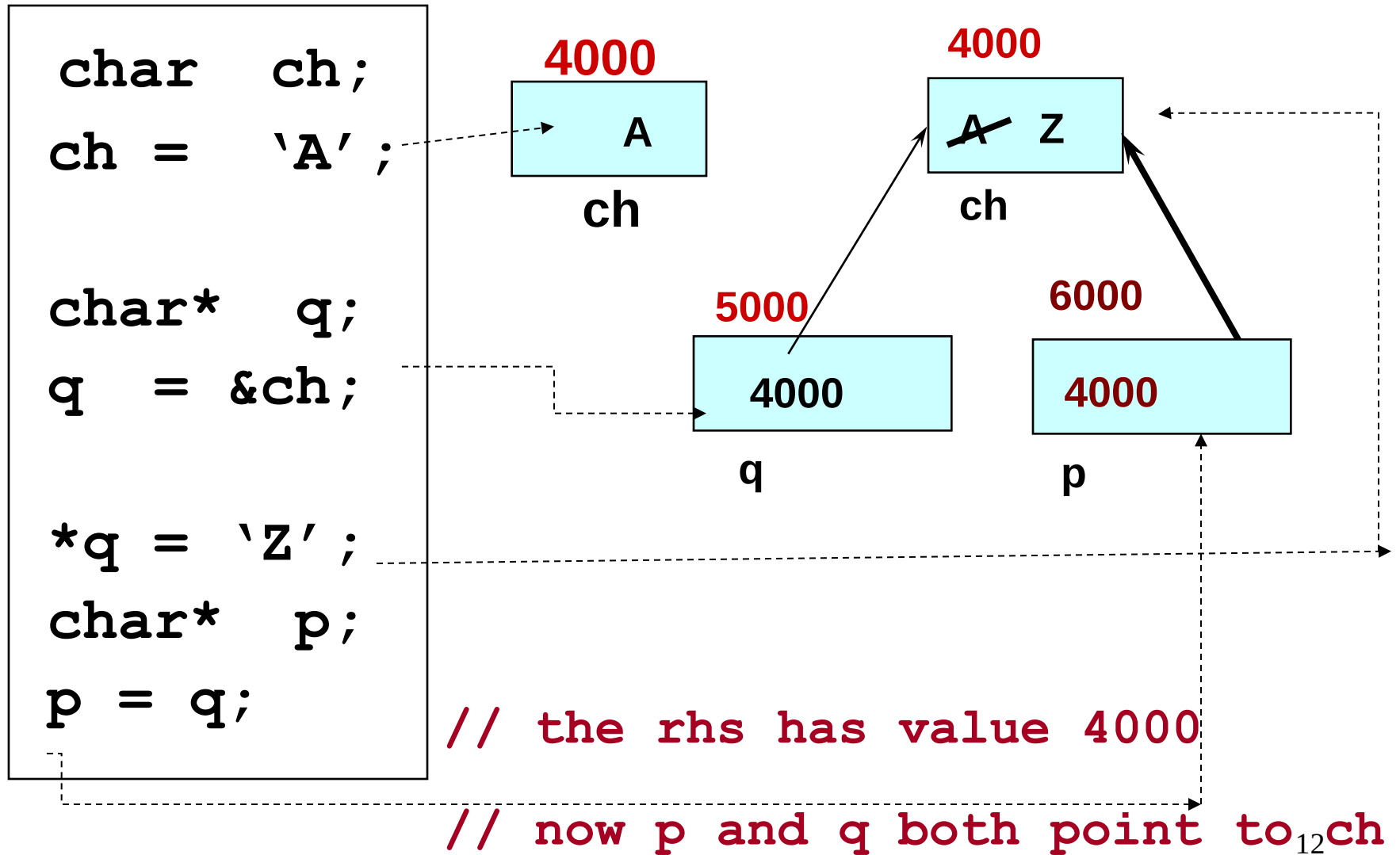
NOTE: The value pointed to by **ptr** is denoted by ***ptr**

* (dereference operator) การ access ข้อมูลที่ pointer ชี้อยู่

Using the Dereference Operator



Self –Test on Pointers



```
int x;  
int *p = &x; *q = &x;
```

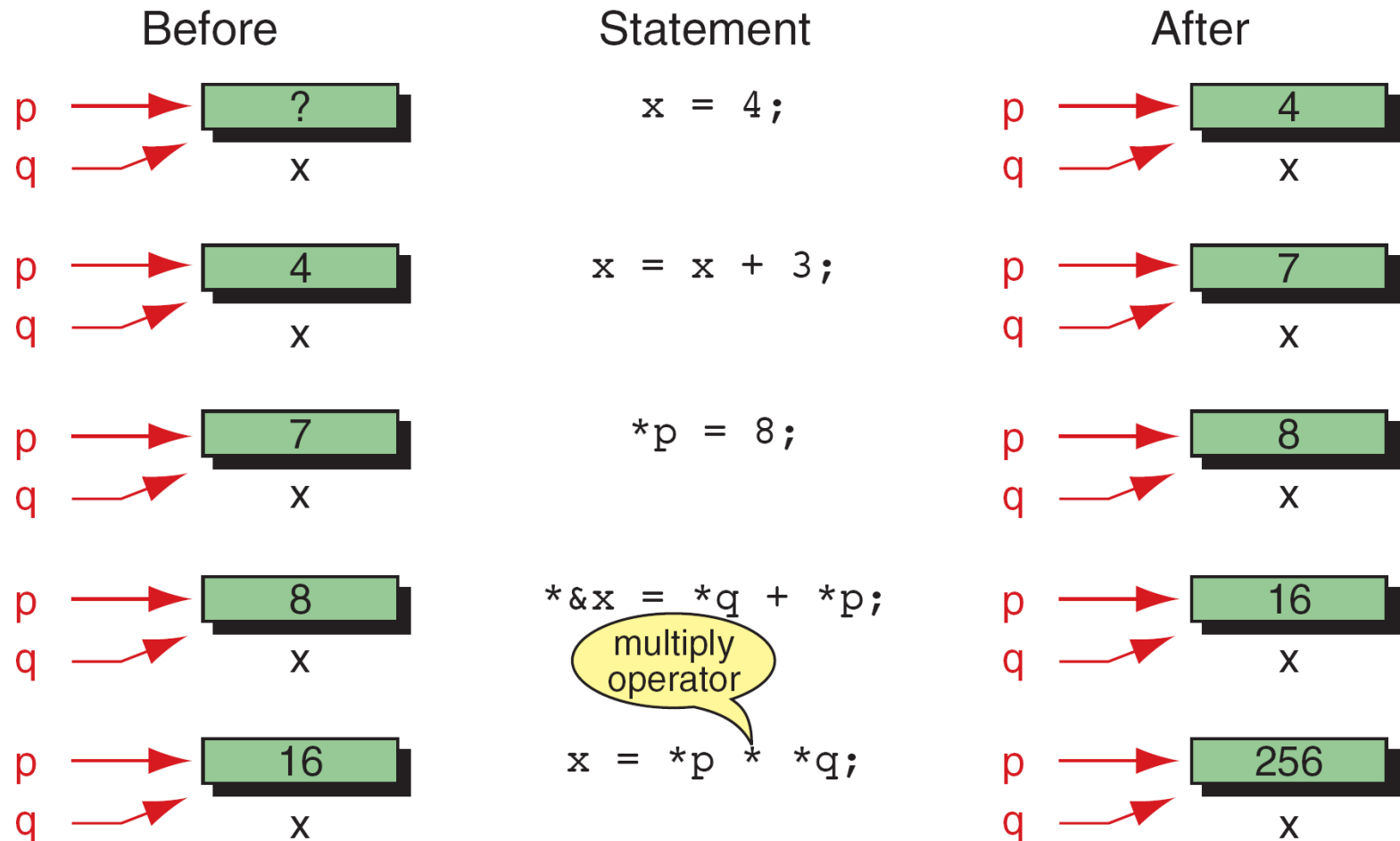
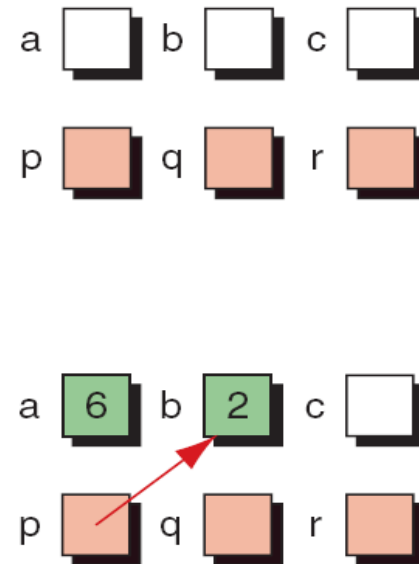


FIGURE 9-8 Accessing Variables Through Pointers

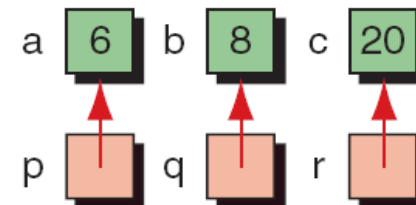
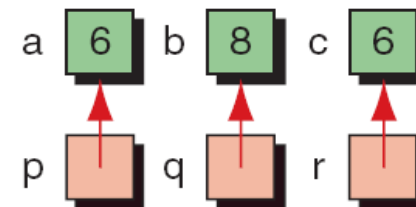
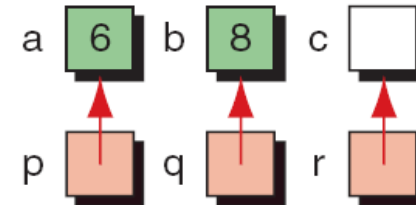
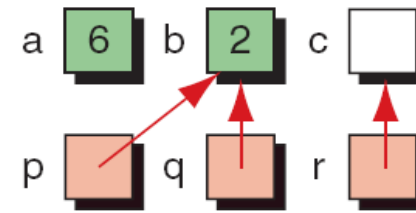
PROGRAM 9-2 Fun with Pointers

```
1  /* Fun with pointers
2      Written by:
3      Date:
4  */
5  #include <stdio.h>
6
7  int main (void)
8  {
9      // Local Declarations
10     int  a;
11     int  b;
12     int  c;
13     int* p;
14     int* q;
15     int* r;
16
17     // Statements
18     a = 6;
19     b = 2;
20     p = &b;
21
```



PROGRAM 9-2 Fun with Pointers

```
22     q = p;  
23     r = &c;  
24  
25     p = &a;  
26     *q = 8;  
27  
28     *r = *p;  
29  
30     *r = a + *q + *&c;  
31  
32     printf("%d %d %d \n",  
33           a, b, c);
```



PROGRAM 9-2 Fun with Pointers

```
34     printf("%d %d %d",  
35             *p, *q, *r);  
36     return 0;  
37 } // main
```

Results:

6 8 20

6 8 20

Structure : Pointer

If `a` is a pointer to a struct, then to access the struct's members, we use the `->` operator as in `a->x` or `(*a).x`

main.cpp

```
1  #include <iostream>
2  #include <stdio.h>
3  using namespace std;
4  struct point {
5      int x;
6      int y;
7  };
8
9  int main()
10 {
11     struct point p1 = { 20,30 }, *p2;
12     p2 = &p1;
13
14     printf("\n p1.x = %d , p1.y = %d", p1.x, p1.y);
15     printf("\n p2->x = %d , p2->y = %d", p2->x, p2->y);
16     printf("\n (*p2).x = %d , (*p2).y = %d", (*p2).x, (*p2).y);
17
18 }
19
```

```
p1.x = 20 , p1.y = 30
p2->x = 20 , p2->y = 30
(*p2).x = 20 , (*p2).y = 30
```

```

1  #include <iostream>
2  #include <stdio.h>
3  using namespace std;
4  struct point {
5      int x;
6      int y;
7  };
8
9  struct point getinput() {
10     struct point p1 = { 20,30 }, *p2;
11     return p1;
12 }
13
14 void output( struct point p1 ,struct point *p2 ){
15     printf("\n p1.x = %d , p1.y = %d", p1.x,  p1.y);
16     printf("\n p2->x = %d , p2->y = %d", p2->x,  p2->y);
17     printf("\n (*p2).x = %d , (*p2).y = %d", (*p2).x,  (*p2).y);
18 }
19
20 int main()
21 {
22     struct point p1 = getinput();
23     output( p1, &p1);
24 }
25

```

```

p1.x = 20 , p1.y = 30
p2->x = 20 , p2->y = 30
(*p2).x = 20 , (*p2).y = 30

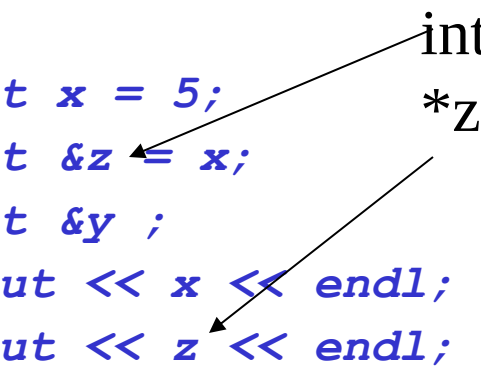
```

Reference Variables (reference type)

Reference variable = *alias for another variable*

- Contains the address of a variable (like a pointer)
- No need to perform any dereferencing (unlike a pointer)
- Must be initialized when it is declared

```
int *z = &x
int x = 5;
int &z = x; // z is another name for x
int &y ; //Error: reference must be initialized
cout << x << endl; -> prints 5
cout << z << endl; -> prints 5
```



➡ `z = 9;` // same as `x = 9;`

```
cout << x << endl; -> prints 9
cout << z << endl; -> prints 9
```

Why Reference Variables

- Are primarily used as function parameters
- Advantages of using references:
 - you don't have to pass the address of a variable
 - you don't have to dereference the variable inside the called function

Reference Variables Example

```
#include <iostream>
using namespace std;
// Function prototypes
// (required in C++)

void p_swap(int *, int *);
void r_swap(int&, int&);

int main (void){
    int v = 5, x = 10;
    cout << v << x << endl;
    p_swap(&v, &x);
    cout << v << x << endl;
    r_swap(v, x);
    cout << v << x << endl;
    return 0;
}
```

```
void p_swap(int *a, int *b)
{
    int temp;
    temp = *a;      (2)
    *a = *b;        (3)
    *b = temp;
}
```

```
void r_swap(int &a, int &b)
{
    int temp;
    temp = a;      (2)
    a = b;         (3)
    b = temp;
}
```

Pointer Arithmetic and Arrays

```
char str [ 8 ];
```

- **str** is the **base address** of the array.
- We say **str** is **a pointer** because its **value is an address**.
- It is **a pointer constant** because the value of **str** itself cannot be changed by assignment. It “points” to the memory location of a **char**.

6000

'H'	'e'	'l'	'l'	'o'	'\0'		
str [0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]

&str[0] = ? ↙ address
&str[2] = ? ↘

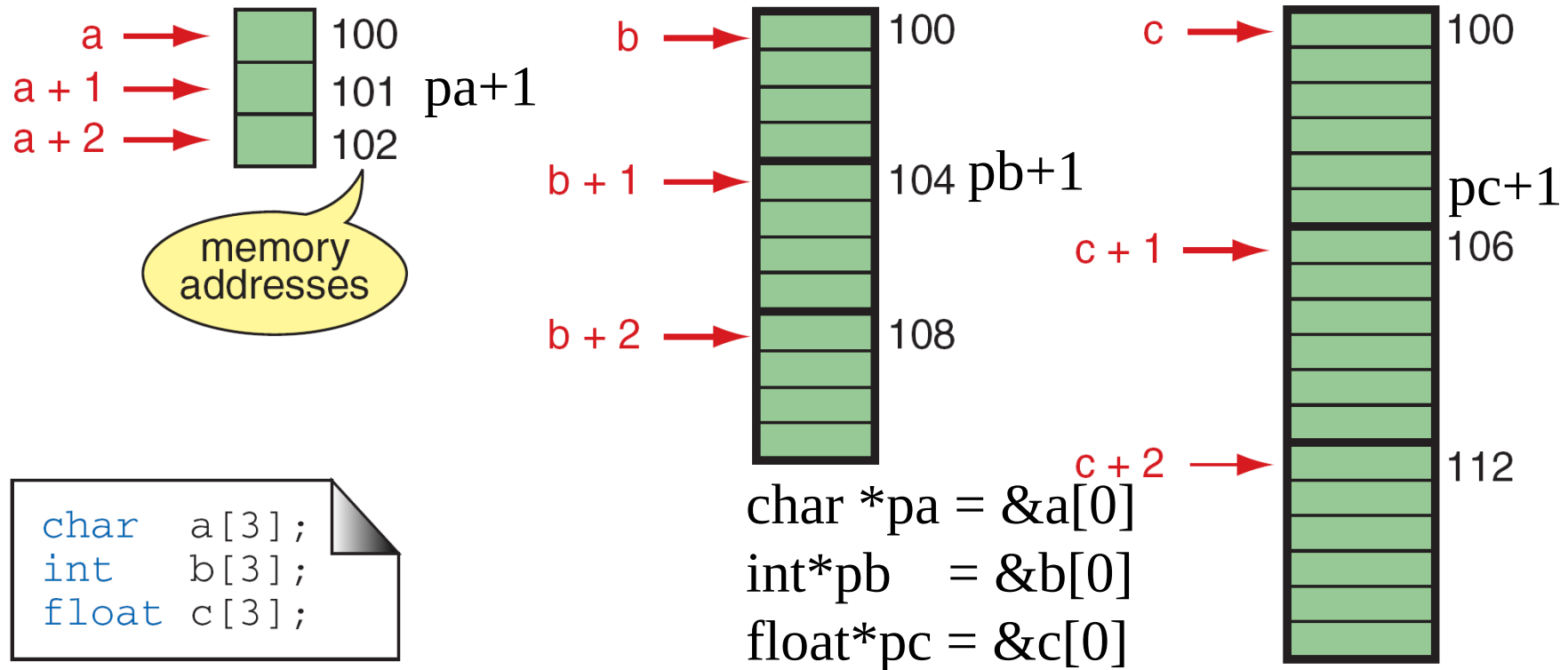
Pointer Arithmetic

Size of memory of pointer data type

```
2  #include <stdio.h>
3
4  using namespace std;
5
6  struct employee {
7      char name[16];
8      int id;
9  };
10
11 int main()
12 {
13     int i, *pi;
14     float f, *pf;
15     char c, *pc;
16     double d, *pd;
17     struct employee e, *pe;
18
19     cout << "\n sizeof(i) = " << sizeof(i) << " sizeof(pi) = " << sizeof(pi) ;
20     cout << "\n sizeof(f) = " << sizeof(f) << " sizeof(pf) = " << sizeof(pf) ;
21     cout << "\n sizeof(c) = " << sizeof(c) << " sizeof(pc) = " << sizeof(pc) ;
22     cout << "\n sizeof(d) = " << sizeof(d) << " sizeof(pd) = " << sizeof(pd) ;
23     cout << "\n sizeof(e) = " << sizeof(e) << " sizeof(pe) = " << sizeof(pe) ;
24
25     return 0;
26 }
```

sizeof(i) = 4 sizeof(pi) = 8
sizeof(f) = 4 sizeof(pf) = 8
sizeof(c) = 1 sizeof(pc) = 8
sizeof(d) = 8 sizeof(pd) = 8
sizeof(e) = 20 sizeof(pe) = 8

Given pointer, p , $p \pm n$ is a pointer to the value n elements away.



$$p + n = \text{address_of_p} + \text{sizeof}(\text{type_of_p})$$

FIGURE 10-6 Pointer Arithmetic and Different Types

```

1  #include <iostream>
2  #include <stdio.h>
3
4  using namespace std;
5
6  struct student {
7      char name[12];
8      int id;
9  };
10
11 int main()
12 {
13     char str[] = "0123456789ABCDEF";
14     char *pc = (char*)str;
15     int *pi = (int*)str;
16     double *pf = (double*)str;
17     struct student *pe = (struct student *)str;
18
19     printf( "\nstr = %p ,str+1 = %p ,str +2 = %p ,sizeof(str) = %d", str,str+1,str+2, sizeof(str));
20     printf( "\npc = %p ,pc+1 = %p ,pc +2 = %p ,sizeof(char) = %d", pc,pc+1,pc+2 , sizeof(char));
21     printf( "\npi = %p ,pi+1 = %p ,pi +2 = %p ,sizeof(int) = %d", pi,pi+1,pi+2, sizeof(int));
22     printf( "\npf = %p ,pf+1 = %p ,pf +2 = %p ,sizeof(double) = %d", pf,pf+1,pf+2, sizeof(double));
23     printf( "\npe = %p ,pe+1 = %p ,pe +2 = %p ,sizeof(struct student) = %d", pe,pe+1,pe+2, sizeof(struct student));
24
25     return 0;
26 }

```

```

str = 0x7ffc76a18eb0 ,str+1 = 0x7ffc76a18eb1 ,str +2 = 0x7ffc76a18eb2 ,sizeof(str) = 17
pc = 0x7ffc76a18eb0 ,pc+1 = 0x7ffc76a18eb1 ,pc +2 = 0x7ffc76a18eb2 ,sizeof(char) = 1
pi = 0x7ffc76a18eb0 ,pi+1 = 0x7ffc76a18eb4 ,pi +2 = 0x7ffc76a18eb8 ,sizeof(int) = 4
pf = 0x7ffc76a18eb0 ,pf+1 = 0x7ffc76a18eb8 ,pf +2 = 0x7ffc76a18ec0 ,sizeof(double) = 8
pe = 0x7ffc76a18eb0 ,pe+1 = 0x7ffc76a18ec0 ,pe +2 = 0x7ffc76a18ed0 ,sizeof(struct student) = 16

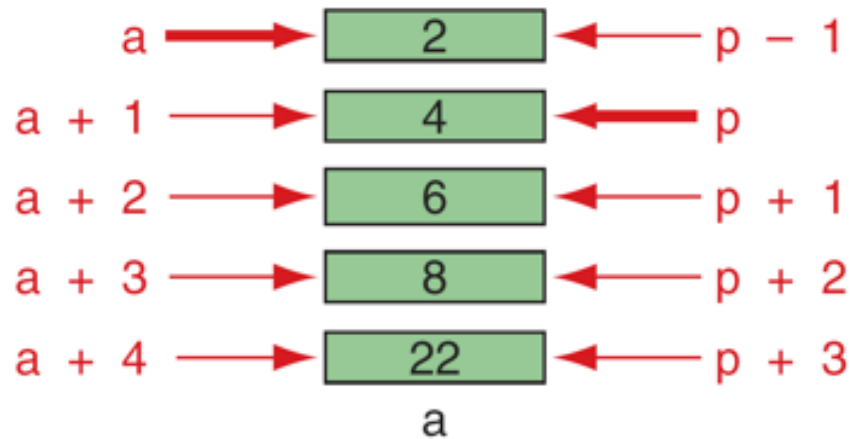
```

...Program finished with exit code 0

Pointer Arithmetic and Arrays

Given pointer , p , $p \pm n$ is a pointer to the value n elements away.

Concept: Suppose $p = \&a[1]$



Pointer Arithmetic and Arrays

- Let re-cap, if an array named `iListArray[ ]` is a declared array, the expression `* iListArray` is the array's first element, `* ( iListArray + 1)` is the array's second element, and so on.
- Generally, the relationship is,

```
* (iListArray)    ==  iListArray[0]    // first element
* (iListArray + 1) ==  iListArray[1]    // second element
* (iListArray + 2) ==  iListArray[2]    // third element
...
...
* (iListArray + n) ==  iListArray[n]    // the nth element
```

- So, you can see the equivalence of array subscript notation and array pointer notation.

Pointer Arithmetic and Arrays

```
char msg[ ] = "Hello";
```

```
char* ptr;
```

```
ptr = msg;
```

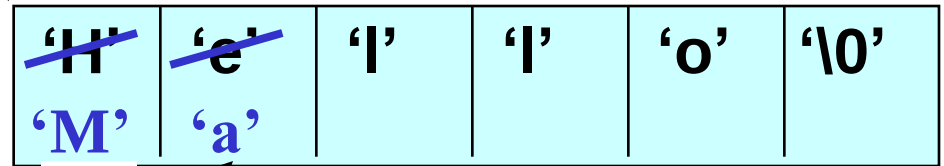
```
*ptr = 'M' ;
```

```
ptr++;
```

```
*ptr = 'a';
```

msg

3000



3001

ptr

&msg[0] = ?

&msg[3] = ?

NOTE:C/C++ array of char vs pointer string (string constant)

main.cpp

```
1  #include <iostream>
2  #include <stdio.h>
3  using namespace std;
4  int main()
5  {
6      char str0[] = "0123456789ABCDEF";
7      char *str1  = "0123456789ABCDEF";
8
9      printf( "\nstr0 = %s ,sizeof(str0) = %d",
10             str0, sizeof(str0));
11      printf( "\nstr1 = %s ,sizeof(str1) = %d , sizeof(char*) = %d",
12             str1, sizeof(str1) , sizeof(char*));
13
14      return 0;
15  }
16
```

str0 = 0123456789ABCDEF ,sizeof(str0) = 17

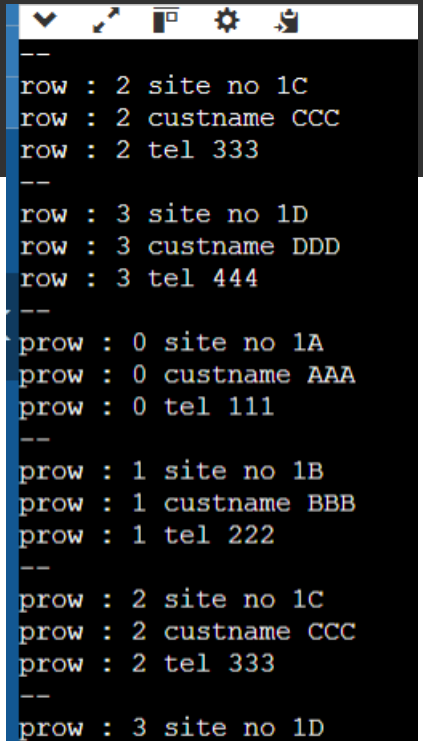
str1 = 0123456789ABCDEF ,sizeof(str1) = 8 , sizeof(char*) = 8

```
7 int main()
8 {
9     char str0[] = "0123456789ABCDEF";
10    char *str1 = "0123456789ABCDEF";
11
12    printf( "\nstr0 = %s ,sizeof(str0) = %d", str0, sizeof(str0));
13    printf( "\nstr1 = %s ,sizeof(str1) = %d , sizeof(char*) = %d", str1, sizeof(str1) , sizeof(char*));
14
15
16
17    str0[2] = 'b';
18    cout << "\nstr0[2] -> " << str0[2] ;
19    // str0++;
20    str1++;
21    cout << "\n*(str1) -> " << *(str1) ;
22    *(str1+2) = 'B';
23    cout << "\n*(str1+2) -> " << *(str1+2);
24
25    return 0;
26 }
27
28
```

Array of Structure & pointer

```
6 struct Seat
7 {
8     char siteno[5];
9     char custname[50];
10    int tel;
11 };
12
13 int main()
14 {
15     struct Seat site[4] = {
16         {"1A", "AAA", 111},
17         {"1B", "BBB", 222},
18         {"1C", "CCC", 333},
19         {"1D", "DDD", 444}
20     };
21
22     struct Seat *psite = site;
23     //struct Seat *psite = &site[0];
24 }
```

```
25 for(int i=0; i < 4; i++) {
26     cout << "\nrow : " << i << " site no " << site[i].siteno ;
27     cout << "\nrow : " << i << " custname " << site[i].custname ;
28     cout << "\nrow : " << i << " tel " << site[i].tel ;
29     cout << "\n--";
30 }
31
32 for(int i=0; i < 4; i++) {
33     cout << "\nrow : " << i << " site no " << psite->siteno ;
34     cout << "\nrow : " << i << " custname " << psite->custname ;
35     cout << "\nrow : " << i << " tel " << psite->tel ;
36     cout << "\n--";
37     psite++;
38 }
39
40 return 0;
41 }
```



```
--
row : 2 site no 1C
row : 2 custname CCC
row : 2 tel 333
--
row : 3 site no 1D
row : 3 custname DDD
row : 3 tel 444
--
row : 0 site no 1A
row : 0 custname AAA
row : 0 tel 111
--
row : 1 site no 1B
row : 1 custname BBB
row : 1 tel 222
--
row : 2 site no 1C
row : 2 custname CCC
row : 2 tel 333
--
row : 3 site no 1D
```

Structure : Itself pointer

```
10
11 int main() {
12     struct node a = {'a',10,20 };
13     struct node b = {'b',11,21 };
14     struct node c = {'c',12,22 };
15     struct node d = {'d',13,23 };
16     a.next = &b;
17     b.next = &c;
18     c.next = &d;
19     d.next = NULL;
20     struct node *p = &a;
21
22     printf("p \t\t\t d \t\t x \t\t y \t\t next \n");
23
24     while(p!=NULL) {
25
26         printf("%p \t\t\t %c \t\t %d \t\t %d \t\t %p \n",
27             p,p->d,p->x,p->y,p->next);
28
29         p = p->next;
30     }
31
32     return 0;
33 }
```

```
4
5 struct node {
6     char d;
7     int x,y;
8     struct node *next;
9 };
10
```

p	d	x	y	next
0x7ffc98d34ac0	a	10	20	0x7ffc98d34ae0
0x7ffc98d34ae0	b	11	21	0x7ffc98d34b00
0x7ffc98d34b00	c	12	22	0x7ffc98d34b20
0x7ffc98d34b20	d	13	23	(nil)

```

10
11 int main() {
12     struct node a = {'a',10,20 };
13     struct node b = {'b',11,21 };
14     struct node c = {'c',12,22 };
15     struct node d = {'d',13,23 };
16     a.next = &b;
17     b.next = &c;
18     c.next = &d;
19     d.next = NULL;
20     struct node *p = &a;
21
22     printf("p \t\t\t d \t\t x \t\t y \t\t next \n");
23
24     while(p!=NULL) {
25
26         printf("%p \t\t\t %c \t\t %d \t\t %d \t\t %p \n",
27             p,p->d,p->x,p->y,p->next);
28
29         p = p->next;
30     }
31
32     return 0;
33 }

```

declare

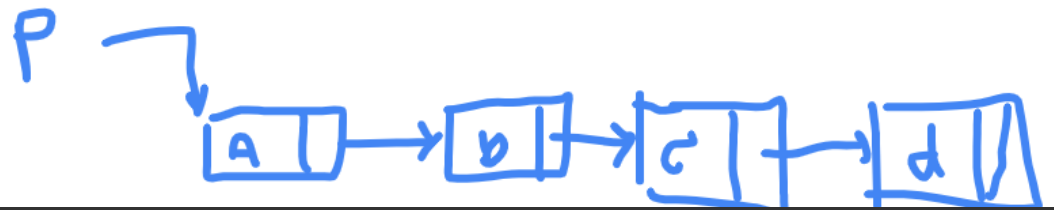
addr	data	structure	variable
1000	'a'	d	
1001	10	x	a
1002	20	y	
1003	?	next	a->next = &b
1004	'b'	d	
1005	11	x	b
1006	21	y	
1007	?	next	b->next = &c
1008	'c'	d	
1009	12	x	c
1010	22	y	
1011	?	next	c->next = &d
1012	'd'	d	
1013	13	x	d
1014	23	y	
1015	?	next	d->next = NULL
1016	?		p

p = &a

Execute

addr	data	structure	variable
1000	'a'	d	
1001	10	x	a
1002	20	y	
1003	1004	next	
1004	'b'	d	
1005	11	x	b
1006	21	y	
1007	1008	next	
1008	'c'	d	
1009	12	x	c
1010	22	y	
1011	1012	next	
1012	'd'	d	
1013	13	x	d
1014	23	y	
1015	NULL	next	
1016	1000		p

Phy vs Logical



addr	data	structure	variable
1000	'a'	d	
1001	10	x	a
1002	20	y	
1003	1004	next	
1004	'b'	d	
1005	11	x	b
1006	21	y	
1007	1008	next	
1008	'c'	d	
1009	12	x	c
1010	22	y	
1011	1012	next	
1012	'd'	d	
1013	13	x	d
1014	23	y	
1015	NULL	next	
1016	1000		p

```

struct node {
    char d;
    int x,y;
    struct node *next;
};

int main() {
    struct node a = {'a',10,20 };
    struct node b = {'b',11,21 };
    struct node c = {'c',12,22 };
    struct node d = {'d',13,23 };
    a.next = &b;
    b.next = &c;
    c.next = &d;
    d.next = NULL;
    struct node *p = &a;

    printf("p \t\t\t\t d \t\t\t\t x \t\t\t\t y \t\t\t\t next \n");

    while(p!=NULL) {

        printf("%p \t\t\t\t %c \t\t\t\t %d \t\t\t\t %d \t\t\t\t %p \n",
            p,p->d,p->x,p->y,p->next);

        p = p->next;
    }

    return 0;
}
  
```

Parameter passing : Array vs Pointer

```
7
8 // "site" is an array data type
9 void showArray(int ary[], int n)
10 {
11     for(int i=0; i < n; i++) {
12         cout << "\nrow ary[i] : " << i << " data " << ary[i] ;
13     }
14
15     for(int i=0; i < n; i++) {
16         cout << "\nrow *ary: " << i << " data " << *ary ;
17         ary++;
18     }
19 }
20
21 // "psite" is pointer
22 void showPointer(int *pary, int n)
23 {
24     for(int i=0; i < n; i++) {
25         cout << "\nrow pary[i] : " << i << " data " << pary[i] ;
26     }
27
28     for(int i=0; i < n; i++) {
29         cout << "\nrow *pary : " << i << " data " << *pary ;
30         pary++;
31     }
32
33 }
```

```
35 int main()
36 {
37     int data[15] = { 10,11,12,13,14,15};
38     cout << "\nshowArray-1";
39     showArray( data,4 );
40     cout << "\nshowArray-2";
41     showArray( &data[2],2 );
42
43     cout << "\nshowPointer-1";
44     showPointer( data,4 );
45     cout << "\nshowPointer-2";
46     showPointer( &data[2],2 );
47     return 0;
48 }
```

```
row ary[i] : 3 data 13
row *ary: 0 data 10
row *ary: 1 data 11
row *ary: 2 data 12
row *ary: 3 data 13
showArray-2
row ary[i] : 0 data 12
row ary[i] : 1 data 13
row *ary: 0 data 12
row *ary: 1 data 13
showPointer-1
row pary[i] : 0 data 10
row pary[i] : 1 data 11
row pary[i] : 2 data 12
row pary[i] : 3 data 13
row *pary : 0 data 10
row *pary : 1 data 11
row *pary : 2 data 12
row *pary : 3 data 13
showPointer-2
row pary[i] : 0 data 12
row pary[i] : 1 data 13
row *pary : 0 data 12
```

Parameter passing :Array vs Pointer

```
6 struct Seat
7 {
8     char siteno[5];
9     char custname[50];
10    int tel;
11 };
12
13 // "site" is an array data type
14 void showArray(struct Seat asite [],int n )
15 {
16     for(int i=0; i < n; i++) {
17         cout << "\nrow : " << i << " site no " << asite[i].siteno ;
18         cout << "\nrow : " << i << " custname " << asite[i].custname ;
19         cout << "\nrow : " << i << " tel " << asite[i].tel ;
20         cout << "\n--";
21     }
22 }
23
24 // "psite" is pointer
25 void showPointer(struct Seat *psite,int n )
26 {
27     for(int i=0; i < n; i++) {
28         cout << "\nrow : " << i << " site no " << psite->siteno ;
29         cout << "\nrow : " << i << " custname " << psite->custname ;
30         cout << "\nrow : " << i << " tel " << psite->tel ;
31         cout << "\n--";
32         psite++;
33     }
34 }
35
```

```
36 int main()
37 {
38     struct Seat site[4] = {
39         {"1A","AAA",111},
40         {"1B","BBB",222},
41         {"1C","CCC",333},
42         {"1D","DDD",444}
43     };
44
45     cout << "\nshowArray-1";
46     showArray( site,4 );
47     cout << "\nshowArray-2";
48     showArray( &site[2],2 );
49
50     cout << "\nshowPointer-1";
51     showPointer( site,4 );
52     cout << "\nshowPointer-2";
53     showPointer( &site[2],2 );
54     return 0;
55 }
```

Dynamic Memory Allocation

In C and C++, three types of memory are used by programs:

- **Static memory** - where global and static variables live
- **Heap memory** - dynamically allocated at execution time
 - "managed" memory accessed using pointers
- **Stack memory** - used by automatic variables

Static Memory

Global Variables

Static Variables

Heap Memory (or free store)

Dynamically Allocated Memory

(Unnamed variables)

Stack Memory

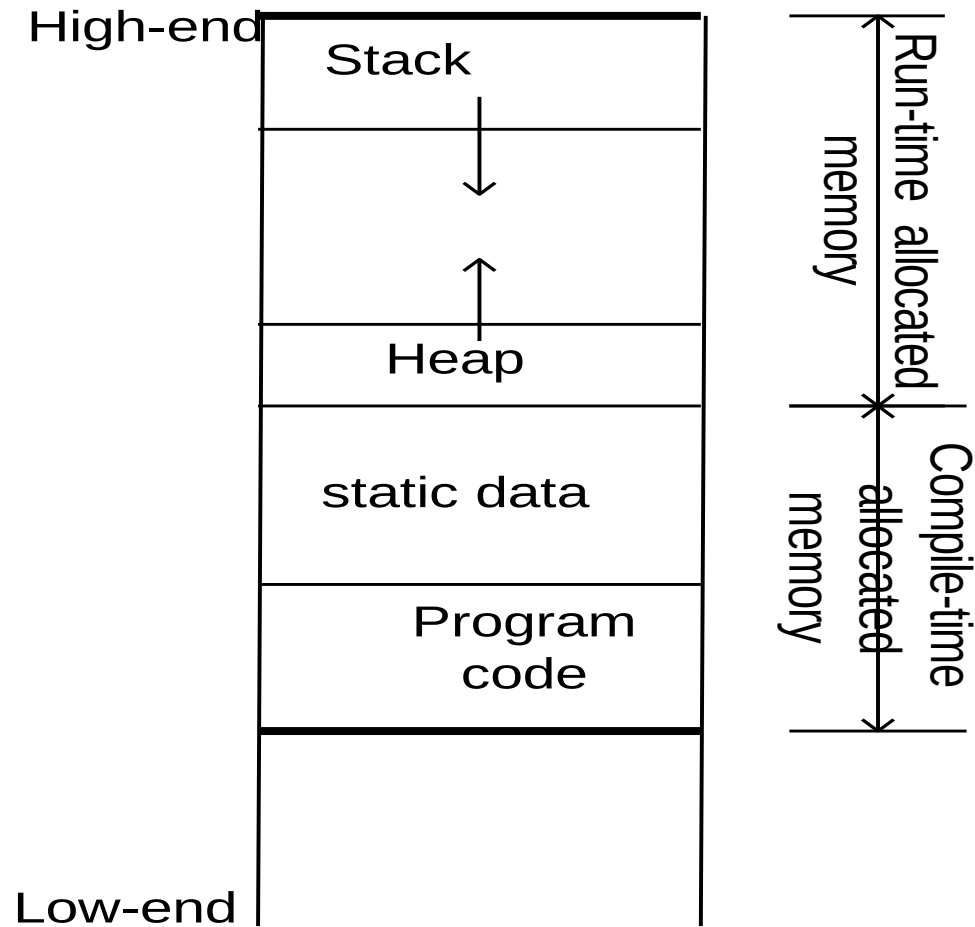
Auto Variables

Function parameters

3 Kinds of Program Data

- **STATIC DATA**: Allocated at compiler time
- **DYNAMIC DATA**: explicitly allocated and deallocated during program execution by C++ instructions written by programmer using operators **new/malloc** and **delete/free**
- **AUTOMATIC DATA**: automatically created at function entry, resides in activation frame of the function, and is destroyed when returning from function

Dynamic Memory Allocation Diagram



Dynamic Memory Allocation

- *In C*, functions such as `malloc()` are used to dynamically allocate memory from the **Heap**.
- *In C++*, this is accomplished using the **new/malloc** and **delete/free** operators
- **new/malloc** is used to allocate memory during execution time
 - returns a pointer to the address where the object is to be stored
 - always returns a pointer to the type that follows the **new /malloc**

Malloc Package

- `#include <stdlib.h>`
- `void *malloc(size_t size)`
 - If successful:
 - Returns a pointer to a memory block of at least `size` bytes, aligned to 8-byte boundary (typically)
 - If `size == 0`, returns `NULL` (design flaw)
 - If unsuccessful: returns `NULL (0)` and sets `errno`
- `void free(void *p)`
 - Returns the block pointed to by `p` to pool of available memory
 - `p` MUST come from a previous call to `malloc` or `realloc`
 - `NULL` not a legal argument (design flaw, sigh)
- `void *realloc(void *p, size_t size)`
 - Changes size of block `p` and returns pointer to new block
 - `NULL` not legal for `p`
 - Contents of new block unchanged up to min of old and new size

Allocating a Block of Memory

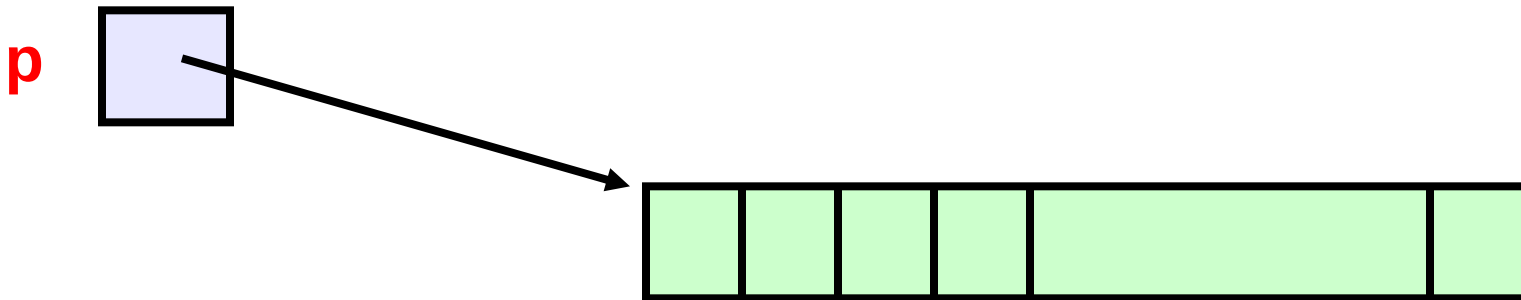
- A block of memory can be allocated using the function `malloc`
 - Reserves a block of memory of specified size and returns a pointer of type `void`
 - The return pointer can be type-casted to any pointer type
- General format:
`type *p;`
`p = (type *) malloc (byte_size);`

Example

`sizeof(int) = 4`

```
int *p = (int *) malloc( 100 * sizeof(int) );
```

- A memory space equivalent to 100 times the size of an int bytes is reserved
- The address of the first byte of the allocated memory is assigned to the pointer `p` of type `int`



400 bytes of space

Contd.

- `cptr = (char *) malloc (20);` `cptr = (char *) malloc (20*sizeof(char));`

Allocates 20 bytes of space for the pointer `cptr` of type `char`

- `sptr = (struct stud *) malloc( 10*sizeof(struct stud) );`

Allocates space for a structure array of 10 elements.
`sptr` points to a structure element of type `struct stud`

Always use `sizeof` operator to find number of bytes for a data type, as it can vary from machine to machine

Points to Note

- `malloc` always allocates a block of contiguous bytes
 - The allocation can fail if sufficient contiguous memory space is not available
 - If it fails, `malloc` returns `NULL` NULL = 0

```
if ((p = (int *) malloc( 100 * sizeof(int) )) == NULL) {  
    printf (“\n Memory cannot be allocated”);  
    exit();  
}
```

Using the malloc'd Array

- Once the memory is allocated, it can be used with pointers, or with array notation
- Example:

```
int *p, n, i ;  
scanf( "%d", &n );  
p = (int *) malloc ( n * sizeof(int) );  
for ( i=0 ; i < n ; ++i )  
    scanf( "%d", &p[i] );
```

The `n` integers allocated can be accessed as `*p, *(p+1), *(p+2), ..., *(p+n-1)` or just as `p[0], p[1], p[2], ..., p[n-1]`

Releasing the Allocated Space: `free`

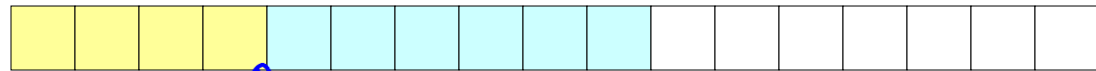
- An allocated block can be returned to the system for future use by using the `free` function
- General syntax:
`free (ptr);`
where `ptr` is a pointer to a memory block which has been previously created using `malloc`
- Note that no size needs to be mentioned for the allocated block, the system remembers it for each pointer returned

Allocation Examples

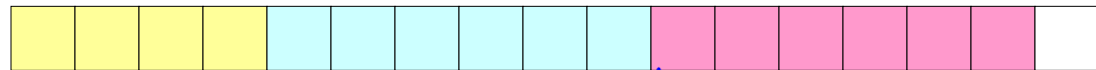
`p1 = malloc(4)` *words*



p1 ↑
`p2 = malloc(5)`

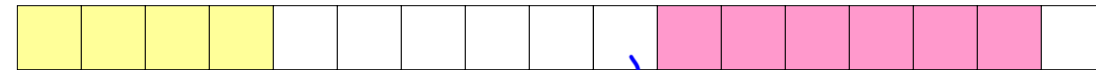


↑ *p2*
`p3 = malloc(6)`



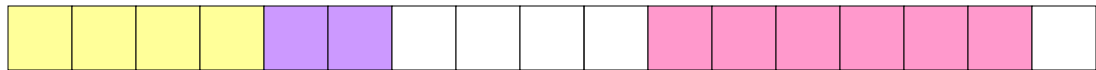
`free(p2)`

↑ *p3.*



`p4 = malloc(2)`

free.

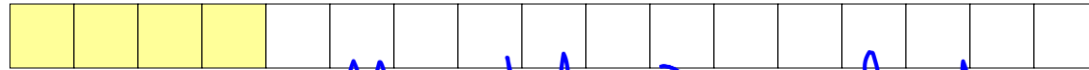


↑ *p4.*

External Fragmentation

Occurs when there is enough aggregate heap memory, but no single free block is large enough

```
p1 = malloc(4)
```

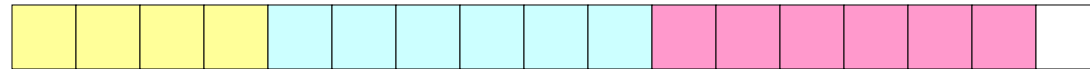


```
p2 = malloc(5)
```

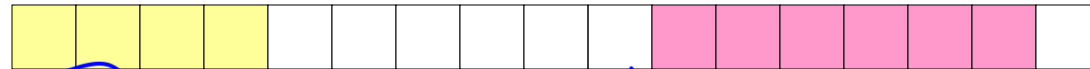


allocated size including metadata should be even

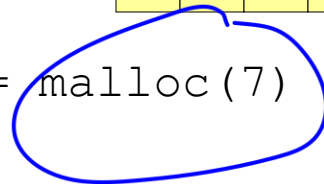
```
p3 = malloc(6)
```



```
free(p2)
```



```
p4 = malloc(7)
```



6 oops!

+1

External fragmentation depends on the pattern of *future* requests, and thus is difficult to measure.

Example

*(height+i)

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main()
5  {
6      int i,N;
7      float *height;
8      float sum=0,avg;
9
10     printf("Input number of students : ");
11     scanf("%d", &N);
12
13     height = (float *)
14         malloc(N * sizeof(float));
15
16     for (i=0; i<N; i++) {
17         printf("Input heights for %d students \n", i+1 );
18         scanf ("%f", &height[i]);
19     }
20
21     for(i=0;i<N;i++)
22         sum += height[i];
23
24     avg = sum / (float) N;
25
26     printf("Average height = %f \n",
27         avg);
28     free (height);
29     return 0;
30 }
```

Can we allocate only arrays?

- `malloc` can be used to allocate memory for single variables also
 - `p = (int *) malloc ( sizeof(int) );`
 - Allocates space for a single int, which can be accessed as `*p`
- Single variable allocations are just special case of array allocations
 - Array with only one element

malloc()-ing array of structures

```
2  #include <stdlib.h>
3
4  typedef struct{
5      char name[20];
6      int roll;
7      float SGPA[2], CGPA;
8  } person;
9  int main()
10 {
11     person *student;
12     int i, j, n;
13     printf("input number of person (n): ");
14     scanf("%d", &n);
15     student = (person *)malloc( n*sizeof(person) );
16     for ( i=0 ; i<n ; i++ ) {
17         printf("input name (s): ");
18         scanf( "%s", student[i].name );
19         printf("input roll (n): ");
20         scanf( "%d", &student[i].roll );
21         for( j=0; j<2 ; j++ ) {
22             printf("input SGPA [%d] (f): " , j+1);
23             scanf("%f", &student[i].SGPA[j] );
24         }
25         printf("input CPGA (f) : ");
26         scanf("%f", &student[i].CGPA);
27         printf("---- \n");
28     }
29     free (student);
30 }
```

array of pointers

Output

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5
6 int main()
7 {
8     char word[ 30 ], *w[ 3 ];
9     int i, n;
10    printf( "get number of word : ");
11    scanf("%d",&n);
12    printf( "Pls, word \n");
13    for (i=0; i<n; ++i) {
14        printf( "word : ");
15        scanf( "%s", word );
16        w[i] = (char *) malloc ( (strlen(word)+1)*sizeof(char) );
17        strcpy ( w[i], word ) ;
18    }
19    printf( "show word list \n");
20    for (i=0; i<n; i++) {
21        printf("w[%d] = %s    , len = %d \n",i,w[i] , strlen(w[i]) ) ;
22        free ( w[ i ] ) ;
23    }
24    return 0;
25 }
```

free(w) -> ผิด

4

Tendulkar

Sourav

Khan

India

w[0] = Tendulkar

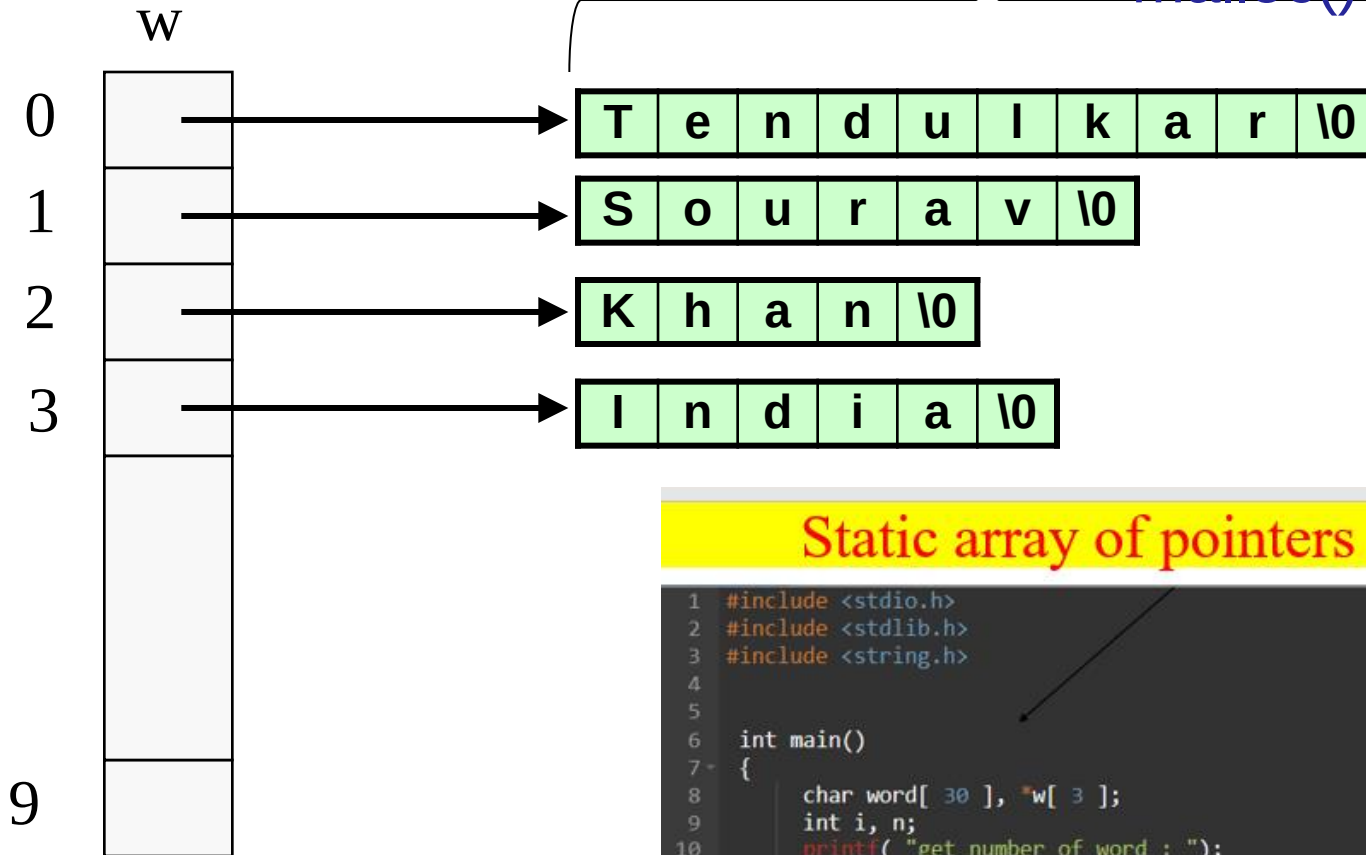
w[1] = Sourav

w[2] = Khan

w[3] = India

How it will look like

malloc()



Static array of pointers

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5
6 int main()
7 {
8     char word[ 30 ], *w[ 3 ];
9     int i, n;
10    printf( "get number of word : " );
11    scanf( "%d", &n );
12    printf( "Pls, word \n" );
13    for ( i=0; i<n; ++i ) {
14        printf( "word : " );
15        scanf( "%s", word );
16        w[i] = (char *) malloc ( (strlen(word)+1)*sizeof(char) );
17        strcpy ( w[i], word );
18    }
19    printf( "show word list \n" );
20    for ( i=0; i<n; i++ ) {
21        printf( "w[%d] = %s , len = %d \n", i, w[i], strlen(w[i]) );
22        free ( w[i] );
23    }
24    return 0;
25 }
```

Output

```
4
Tendulkar
Sourav
Khan
India

w[0] = Tendulkar
w[1] = Sourav
w[2] = Khan
w[3] = India
```

New/delete operator on c++

Operator **new** Syntax

new **DataType**

new **DataType** [**IntExpression**]

- If memory is available, in an area called the heap (or free store) **new** allocates the requested object or array, and returns a pointer to (address of) the memory allocated.
- Otherwise, program terminates with error message.
- The dynamically allocated object exists until the delete operator destroys it.

Operator *new*

```
char* ptr;
```

```
ptr = new char;
```

```
*ptr = 'B';
```

```
cout << *ptr;
```

2000

5000

ptr

5000

'B'

NOTE: Dynamic data has no variable name

It return NULL ,when memory is not available.

The **NULL** Pointer

- There is a pointer constant called the “null pointer” denoted by **NULL**
- But **NULL** is not memory address 0.
- **NOTE:** It is an error to dereference a pointer whose value is **NULL**. Such an error may cause your program to crash, or behave erratically. It is the programmer’s job to check for this.

```
while (ptr != NULL) {  
    . . .                // ok to use *ptr here  
}
```

Operator `delete` Syntax

`delete` `Pointer`

`delete` [] `Pointer`

- The **object or array currently pointed to by `Pointer` is deallocated**, and the value of `Pointer` is undefined. The memory is returned to the free store.
- Good idea to set the pointer to the released memory to `NULL`
- Square brackets are used with `delete` to deallocate a dynamically allocated array.

Operator **delete**

```
char* ptr;
```

```
ptr = new char;
```

```
*ptr = 'B' ;
```

```
cout << *ptr;
```



```
delete ptr;
```

2000

???

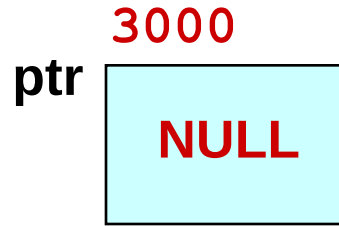
ptr

NOTE:

delete deallocates the
memory pointed to by ptr

Example

```
char *ptr ;  
  
ptr = new char[ 5 ];  
  
strcpy( ptr, "Bye" );  
  
ptr[ 0 ] = 'u';  
  
delete [] ptr;  
  
ptr = NULL;
```



// deallocates the array pointed to by ptr
// ptr itself is not deallocated
// the value of ptr becomes undefined

Pointers and Constants

```
char* p;  
p = new char[20];
```

```
char c[ ] = "Hello";  
const char* pc = c; //pointer to a constant  
pc[2] = 'a'; // error ไม่สามารถกำหนดค่าใหม่ได้  
pc = p;
```

```
char *const cp = c; //constant pointer  
cp[2] = 'a';  
cp = p; // error ไม่สามารถเปลี่ยนค่าได้
```

malloc vs new

```
char *pc = (char *)malloc( sizeof(char)*1 );
*pc = 'A';
cout << "*pc = " << *pc << "\n";
free(pc);
```

```
char *pc = new char;
*pc = 'A';
cout << "*pc = " << *pc << "\n";
delete [] pc;
```

```
char *pc_a = (char *)malloc( sizeof(char)*10 );
strcpy( pc_a, "A0123456");
cout << "*pa = " << pc_a << "\n";
free(pc_a);
```

```
char *pc_a = new char [10];
strcpy( pc_a, "A0123456");
cout << "*pa = " << pc_a << "\n";
delete [] pc_a;
```

malloc vs new

```
int *pi = (int *)malloc( sizeof(int)*1 );
*pi = 10;
cout << "*pi = " << *pi << "\n";
free(pi);
```

```
int *pi = new int;
*pi = 10;
cout << "*pi = " << *pi << "\n";
delete [] pi;
```

```
int *pi_a = (int*)malloc( sizeof(int)*10 );
*(pi_a+0) = 100;
*(pi_a+1) = 200;
cout << "*pi_a[0] = " << pi_a[0] << "\n";
cout << "*pi_a[1] = " << *(pi_a+1) << "\n";
free(pi_a);
```

```
int *pi_a = new int[10];
*(pi_a+0) = 100;
*(pi_a+1) = 200;
cout << "*pi_a[0] = " << pi_a[0] << "\n";
cout << "*pi_a[1] = " << *(pi_a+1) << "\n";
delete [] pi_a;
```

malloc vs new

```
struct employee {  
    int id;  
    char name[10];  
};
```

```
struct employee *e = (struct employee*) malloc(sizeof(struct employee)*1);  
e->id = 10;  
strcpy(e->name, "apirak-1");  
cout << "e->id = " << e->id << ", e->name = " << e->name << "\n";  
free(e);
```

```
struct employee *e = new struct employee;  
e->id = 10;  
strcpy(e->name, "apirak-1");  
cout << "e->id = " << e->id << ", e->name = " << e->name << "\n";  
delete [] e;
```

malloc vs new

```
struct employee {  
    int id;  
    char name[10];  
};
```

```
struct employee *e = (struct employee*) malloc(sizeof(struct employee)*10);  
e->id = 10;  
strcpy(e->name, "apirak-1");  
cout << "e->id = " << e->id << ", e->name = " << e->name << "\n";  
free(e);
```

```
struct employee *e_a = new struct employee[10];  
e_a[0].id = 100;  
strcpy(e_a[0].name, "apirak-100");  
e_a[1].id = 200;  
strcpy(e_a[1].name, "apirak-200");  
cout << "e_a[0].id = " << e_a[0].id << ", e_a[0].name = " << e_a[0].name << "\n";  
cout << "e_a[1].id = " << e_a[1].id << ", e_a[1].name = " << e_a[1].name << "\n";  
delete [] e_a;
```

```

12 int id;
13 char name[10];
14 };
15 int main()
16 {
17     char *pc = (char *)malloc( sizeof(char)*1 );
18     *pc = 'A';
19     cout << "*pc = " << *pc << "\n";
20     free(pc);
21
22     char *pc_a = (char *)malloc( sizeof(char)*10 );
23     strcpy( pc_a, "A0123456");
24     cout << "*pa = " << pc_a << "\n";
25     free(pc_a);
26
27     int *pi = (int *)malloc( sizeof(int)*1 );
28     *pi = 10;
29     cout << "*pi = " << *pi << "\n";
30     free(pi);
31
32     int *pi_a = (int*)malloc( sizeof(int)*10 );
33     *(pi_a+0) = 100;
34     *(pi_a+1) = 200;
35     cout << "*pi_a[0] = " << pi_a[0] << "\n";
36     cout << "*pi_a[1] = " << *(pi_a+1) << "\n";
37     free(pi_a);
38
39
40     struct employee *e = (struct employee*) malloc(sizeof(struct employee)*1);
41     e->id = 10;
42     strcpy(e->name,"apirak-1");
43     cout << "e->id = " << e->id << ",e->name " << e->name << "\n";
44     free(e);
45
46     struct employee *e_a = (struct employee*) malloc(sizeof(struct employee)*10);
47     e_a[0].id = 100;
48     strcpy(e_a[0].name,"apirak-100");
49     e_a[1].id = 200;
50     strcpy(e_a[1].name,"apirak-200");
51     cout << "e_a[0].id = " << e_a[0].id << ",e_a[0].name " << e_a[0].name << "\n";
52     cout << "e_a[1].id = " << e_a[1].id << ",e_a[1].name " << e_a[1].name << "\n";
53     free(e_a);
54
55     return 0;

```

```

*pc = A
*pa = A0123456
*pi = 10
*pi_a[0] = 100
*pi_a[1] = 200
e->id = 10,e->name = apirak-1
e_a[0].id = 100,e_a[0].name = apirak-100
e_a[1].id = 200,e_a[1].name = apirak-200

...Program finished with exit code 0
Press ENTER to exit console.

```

malloc/free

```

11 struct employee {
12     int id;
13     char name[10];
14 };
15 int main()
16 {
17     char *pc = new char;
18     *pc = 'A';
19     cout << "*pc = " << *pc << "\n";
20     delete pc;

21     char *pc_a = new char [10];
22     strcpy( pc_a, "A0123456");
23     cout << "*pa = " << pc_a << "\n";
24     delete [] pc_a;

25

26     int *pi = new int;
27     *pi = 10;
28     cout << "*pi = " << *pi << "\n";
29     delete [] pi;

30

31     int *pi_a =new int[10];
32     *(pi_a+0) = 100;
33     *(pi_a+1) = 200;
34     cout << "*pi_a[0] = " << pi_a[0] << "\n";
35     cout << "*pi_a[1] = " << *(pi_a+1) << "\n";
36     delete [] pi_a;

37

38

39

40     struct employee *e = new struct employee;
41     e->id = 10;
42     strcpy(e->name,"apirak-1");
43     cout << "e->id = " << e->id << ",e->name = " << e->name << "\n";
44     delete [] e;

45

46     struct employee *e_a = new struct employee[10];
47     e_a[0].id = 100;
48     strcpy(e_a[0].name,"apirak-100");
49     e_a[1].id = 200;
50     strcpy(e_a[1].name,"apirak-200");
51     cout << "e_a[0].id = " << e_a[0].id << ",e_a[0].name = " << e_a[0].name << "\n";
52     cout << "e_a[1].id = " << e_a[1].id << ",e_a[1].name = " << e_a[1].name << "\n";
53     delete [] e_a;

54

55     return 0;
56 }
57

```

```

38
*pc = A
*pa = A0123456
*pi = 10
*pi_a[0] = 100
*pi_a[1] = 200
e->id = 10,e->name = apirak-1
e_a[0].id = 100,e_a[0].name = apirak-100
e_a[1].id = 200,e_a[1].name = apirak-200

```

new / delete

Allocate and free other function

malloc/free

```
651 //
652 #include <iostream>
653 #include <string.h>
654 using namespace std;
655
656
657 char* new_arychar(int n) {
658     ... char* pc_a = (char*)malloc(sizeof(char)*n);
659     ... return pc_a;
660 }
661
662 void free_arychar(char* pc) {
663     ... free(pc);
664 }
665
666 int main()
667 {
668     ... char* pc = new_arychar(10);
669     ... *pc = 'A';
670     ... cout << "pc = " << *pc << "\n";
671     ... free_arychar(pc);
672
673 }
```

```

2  #include <iostream>
3  #include <string.h>
4  using namespace std;
5
6
7  int * new_aryint(int n) {
8      int *pi_a = new int[10];
9      return pi_a;
10 }
11
12
13 void free_aryint(int *e_a) {
14     delete [] e_a;
15 }
16
17 int main()
18 {
19
20     int *pi = new_aryint(10);;
21     *pi = 10;
22     cout << "*pi = " << *pi << "\n";
23     free_aryint( pi );
24
25     return 0;
26 }
27

```

new/delete

```

651 //
652 #include <iostream>
653 #include <string.h>
654 using namespace std;
655
656 char * new_arychar(int n) {
657     .... char *pc_a = (char *) malloc( sizeof(char) * n );
658     .... return pc_a;
659 }
660
661 void free_arychar(char * pc) {
662     .... free(pc);
663 }
664
665 int main()
666 {
667     .... char *pc = new_arychar(10);
668     .... *pc = 'A';
669     .... cout << " *pc = " << *pc << "\n";
670     .... free_arychar(pc);
671 }
672
673 }

```

malloc vs new

```

2  #include <iostream>
3  #include <string.h>
4  using namespace std;
5
6
7  int * new_aryint(int n) {
8      int *pi_a = new int[10];
9      return pi_a;
10 }
11
12
13 void free_aryint(int *e_a) {
14     delete [] e_a;
15 }
16
17 int main()
18 {
19
20     int *pi = new_aryint(10);
21     *pi = 10;
22     cout << " *pi = " << *pi << "\n";
23     free_aryint( pi );
24
25     return 0;
26 }
27

```

```

4 struct employee {
5     int id;
6     char name[10];
7 };
8
9 char * new_arychar(int n) {
10     char *pc_a = new char [n];
11     return pc_a;
12 }
13
14 int * new_aryint(int n) {
15     int *pi_a = new int[10];
16     return pi_a;
17 }
18
19 struct employee *new_aryemployee(int n) {
20     struct employee *e_a = new struct employee[n];
21     return e_a;
22 }
23
24 void *free_aryemployee(struct employee *e_a) {
25     delete [] e_a;
26 }

```

new/delete

example

```

28 int main()
29 {
30     char *pc = new_arychar(1);
31     *pc = 'A';
32     cout << "*pc = " << *pc << "\n";
33     delete [] pc;
34
35     char *pc_a = new_arychar(10);
36     strcpy( pc_a, "A0123456");
37     cout << "*pa = " << pc_a << "\n";
38     delete [] pc_a;
39
40     int *pi = new_aryint(1);
41     *pi = 10;
42     cout << "*pi = " << *pi << "\n";
43     delete [] pi;
44
45     int *pi_a = new_aryint(10);
46     *(pi_a+0) = 100;
47     *(pi_a+1) = 200;
48     cout << "*pi_a[0] = " << pi_a[0] << "\n";
49     cout << "*pi_a[1] = " << *(pi_a+1) << "\n";
50     delete [] pi_a;
51
52
53     struct employee *e = new_aryemployee(1);
54     e->id = 10;
55     strcpy(e->name,"apirak-1");
56     cout << "e->id = " << e->id << ",e->name = " << e->name << "\n";
57     delete [] e;
58
59     struct employee *e_a = new_aryemployee(10);
60     e_a[0].id = 100;
61     strcpy(e_a[0].name,"apirak-100");
62     e_a[1].id = 200;
63     strcpy(e_a[1].name,"apirak-200");
64     cout << "e_a[0].id = " << e_a[0].id << ",e_a[0].name = " << e_a[0].name << "\n";
65     cout << "e_a[1].id = " << e_a[1].id << ",e_a[1].name = " << e_a[1].name << "\n";
66     free_aryemployee(e_a);
67
68     return 0;
69 }

```

```

1  #include <iostream>
2  #include <string.h>
3  using namespace std;
4  struct employee {
5      int id;
6      char name[10];
7  };
8
9  char * new_arychar(int n) {
10     char *pc_a = (char *)malloc( sizeof(char)*n );
11     return pc_a;
12 }
13
14 int * new_aryint(int n) {
15     int *pi_a = (int *)malloc( sizeof(int)*n );
16     return pi_a;
17 }
18
19 struct employee *new_aryemployee(int n) {
20     struct employee *e_a =
21         (struct employee *)malloc( sizeof(struct employee)*n );
22     return e_a;
23 }
24
25 void free_aryemployee(struct employee *e_a) {
26     free(e_a);
27 }
28

```

```

29 int main()
30 {
31     char *pc = new_arychar(1);
32     *pc = 'A';
33     cout << "pc = " << *pc << "\n";
34     free(pc);
35
36     char *pc_a = new_arychar(10);
37     strcpy( pc_a, "A0123456");
38     cout << "pa = " << pc_a << "\n";
39     free(pc_a);
40
41     int *pi = new_aryint(1);
42     *pi = 10;
43     cout << "pi = " << *pi << "\n";
44     free(pi);
45
46     int *pi_a = new_aryint(10);
47     *(pi_a+0) = 100;
48     *(pi_a+1) = 200;
49     cout << "pi_a[0] = " << pi_a[0] << "\n";
50     cout << "pi_a[1] = " << *(pi_a+1) << "\n";
51     free(pi_a);
52
53
54     struct employee *e = new_aryemployee(1);
55     e->id = 10;
56     strcpy(e->name, "apirak-1");
57     cout << "e->id = " << e->id << ", e->name = " << e->name << "\n";
58     free(e);
59
60     struct employee *e_a = new_aryemployee(10);
61     e_a[0].id = 100;
62     strcpy(e_a[0].name, "apirak-100");
63     e_a[1].id = 200;
64     strcpy(e_a[1].name, "apirak-200");
65     cout << "e_a[0].id = " << e_a[0].id << ", e_a[0].name = " << e_a[0].name << "\n";
66     cout << "e_a[1].id = " << e_a[1].id << ", e_a[1].name = " << e_a[1].name << "\n";
67     free_aryemployee(e_a);
68 }
69

```

example
malloc/free

ตัวอย่าง

ให้นักศึกษาเขียนโปรแกรมขายสินค้าและพิมพ์ใบเสร็จรับเงิน
โดยข้อมูลในระบบมีไม่เกิน **100** รายการ
ข้อมูลขายสินค้าด้วย

- รหัสสินค้า(id) เช่น “AB001”
 - ชื่อสินค้า(name) เช่น “CAFÉ”
 - ราคาขาย(cost) เช่น 30.6
 - จำนวนสินค้าในสต็อก (instock) เช่น 5

Menu

1. Add item to Cart
2. Sell Product
3. Check Stock
4. End Program

Please select 1 / 2 / 3/ 4 =?

Convert The program to use malloc/new

Rewrite the code using function memory allocation

ตัวอย่าง

ให้นักศึกษาเขียนโปรแกรมขายสินค้าและพิมพ์ใบเสร็จรับเงิน
โดยข้อมูลในระบบมีไม่เกิน **100** รายการ

ข้อมูลขายสินค้าด้วย

- รหัสสินค้า(id) เช่น “AB001”
- ชื่อสินค้า(name) เช่น “CAFÉ”
- ราคาขาย(cost) เช่น 30.6
- จำนวนสินค้าในสต็อก (instock) เช่น 5

Menu

1. Add item to Cart
 2. Sell Product
 3. Check Stock
 4. End Program
- Please select 1 / 2 / 3/ 4 =?

กด 1

Input your product data:

id : **AB001**

name : **CAFE**

Cost : **30.6**

Stock : **5**

กด 2

Input your product id: **AB001**

Input your sell volume : **2**

name : **CAFE**

Cost : **30.6**

stock : **3**

Pay : 61.2 bath

Menu

1. Add item to Cart
2. Sell Product
3. Check Stock
4. End Program

Please select 1 / 2 / 3/ 4 =?

กด 3

product List:

Id	name	cost	stock
----	------	------	-------

AB001	CAFÉ	30.6	3
-------	------	------	---

กด 4

exit from program

```

4
5 using namespace std;
6
7 struct product {
8     char id[10];
9     char name[30];
10    float cost;
11    int instock;
12 };
13
14 int main()
15 {
16     struct product item[100];
17     int n = 0;
18     int select = -1;
19
20     while (true)
21     {
22
23         cout << " Menu" << endl;
24         cout << "1. Add item to Cart " << endl;
25         cout << "2. Sell product " << endl;
26         cout << "3. Check stock " << endl;
27         cout << "4. End program" << endl;
28         cout << "Please select 1 / 2 / 3/ 4 =? ";
29         cin >> select;
30
31         if (select == 4) {
32             break;
33         }
34         switch (select)
35         {
36             case 1:

```

single module Version

แก้ไขให้เป็น dynamic

Multiple function module Version

```
729
730 void report(int n, struct person p[]) {
731     for(int i = 0; i < n; i++) {
732         cout << "id: " << setw(10) << " ";
733         cout << p[i].id << setw(10) << " ";
734         cout << "name: " << setw(10) << " ";
735         cout << p[i].name << setw(10) << " ";
736         cout << "addr: " << setw(10) << " ";
737         cout << p[i].addr << setw(10) << " ";
738         cout << "deposit: " << setw(10) << " ";
739         cout << p[i].deposit << setw(10) << " ";
740         cout << endl;
741     }
742 }
```

```
743
744
745 int main()
746 {
747     struct person customer[500];
748     int n=0;
749     int choice;
750     char id[5];
751     float _deposit, _widthdraw;
752     char name[100];
753     char addr[100];
754     int res;
755     choice = displaymenu();
756     while(choice != 6) {
757         switch(choice) {
758             case 1:
759                 /*
```

```
struct person {
    char id[5];
    char name[30];
    char addr[50];
    float deposit;
};
int displaymenu()
```

แก้ไขให้เป็น dynamic

Problems with Pointers

- **Dangling pointers** (dangerous)
 - A pointer points to a heap-dynamic variable that has been **deallocated (free memory)**
 - **has pointer but no storage**
 - What happen when deferencing a dangling pointer?
- **memory leakage** (Lost heap-dynamic variable)
 - An allocated heap-dynamic variable that is no longer accessible to the user program (often called *garbage*)
 - has storage but no pointer
 - The process of losing **heap-dynamic variables is called *memory leakage***

What does memory leaking mean?

- ▶ Definition:
 - ▶ Particular type of unused memory, unable to release
- ▶ Common:
 - ▶ Refer to any unwanted increase in memory usage
- ▶ Different from memory fragmentation

* for heap memory

```
void Leak()
```

```
{
```

```
    int *A = new int[1000];
```

```
    // some code here
```

```
    // ...
```

```
    // without delete A
```

```
    //
```

```
    return;
```

```
}
```

4000 bytes

Return without free A →
Leak

```

void leak()
{
    int **list = new int*[10];
    for (int i = 0; i < 10; i++)
    {
        list[i] = new int;
    }

    delete list;
    return;
}

```

Leak !

- Allocation a series,
delete only one unit

Solution

```

for (int i = 0; i < 10; i++)
{
    delete list[i];
}
delete list;

```

```

char* GenString()
{
    char *a = new char[10];
    a[9] = '\0';
    return a;
}

void Leak()
{
    char *str = GenString();
    printf("%s\n", str);
    printf("%s\n", GenString());
}

```

Leak memory when
using pointer-return-
type

Solution

```
delete []str;
```

Avoid to call directly GenString()

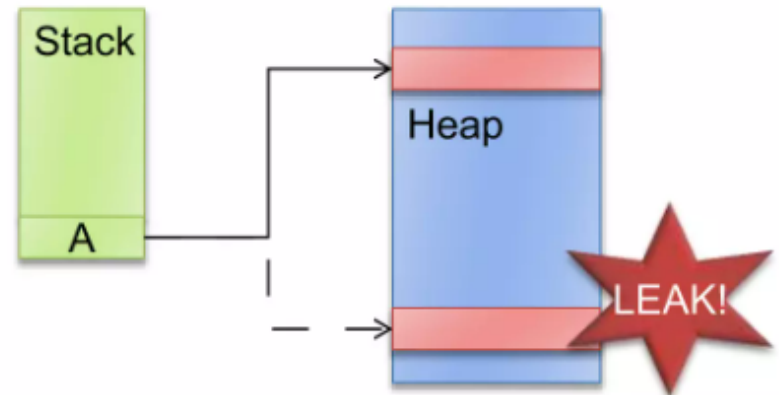
Leak memory because of misunderstanding “=” operator in C++

```
#include <cstdio>

void Foo()
{
    int* a = new int[100];
    a = new int[1000];
}

void Foo1()
{
    int* a = new int[100];
    int* b = new int[1000];
    a = b;
}
```

Leak!



HOMework1 : จากตัวอย่างที่ 1 ให้ นศแปลง โปรแกรมโดยใช้เทคนิค structure

สมมติว่าข้อมูล อาจารย์คณะวิทยาศาสตร์มี n ดังนี้

เพศ (sex) : ชาย = 1 , หญิง = 2

อายุ (age) : ? ปี

สถานภาพสมรส (status) : โสด = 1 , แต่งงาน = 2 , อื่นๆ = 2

ปริญญาสูง (degree) : ตรี = 1 , โท = 2 , เอก = 3

จำนวนปีที่สอน (experience) : ? ปี

Number of teacher = 2

Sex = 1

Age = 35

Status 1

Degree =2

Experient = 10

Sex = 1

Age = 35

Status 1

Degree =2

Experient = 10

จงเขียน

1. โปรแกรมรับข้อมูลจำนวนอาจารย์ n คน ของคณะวิทยาศาสตร์ และข้อมูลของอาจารย์
2. หาจำนวนปีเฉลี่ยของ อายุ และ จำนวนปีที่สอน

AVERAGE AGE = xx.xx Years

AVERAGE Experience = XX.XX Years

Ex 1 จงใช้โครงสร้างข้อมูลแบบ record

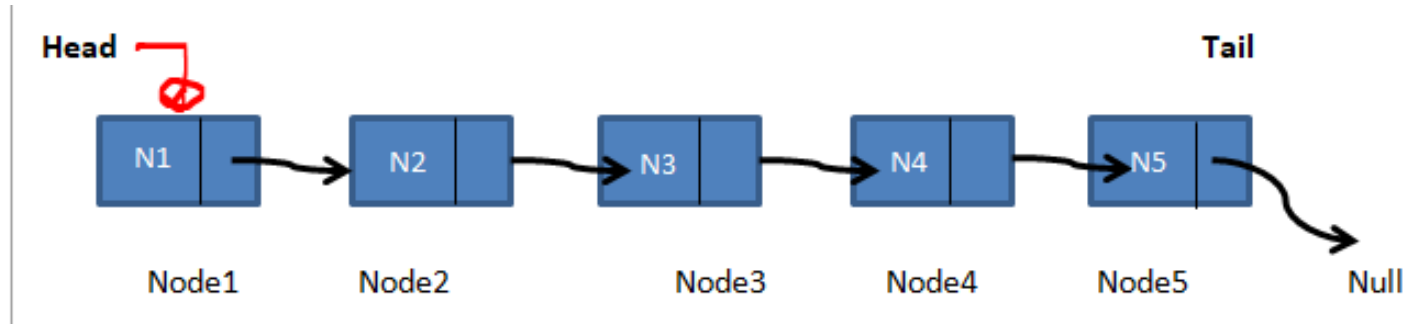
น.ศ. n คน แต่ละคนสอบ 3 ครั้งในวิชาเขียนโปรแกรม P1,P2,P3 และได้คะแนนสอบ 3 ครั้งในวิชาคณิตศาสตร์ (M1,M2,M3)

จงเขียนโปรแกรมรับจำนวนนักศึกษา, ชื่อ, คะแนน, ของแต่ละวิชา ในแต่ละครั้ง เพื่อหาคะแนนเฉลี่ยของ น.ศ. แต่ละคนแต่ละวิชา รวมทั้งคนที่มีคะแนนเฉลี่ยวิชาคณิตศาสตร์สูงสุด

น.ศ. คนที่ 1	20	30	20	คณิตศาสตร์
	40	50	20	เขียนโปรแกรม
น.ศ. คนที่ 2	35	30	25	คณิตศาสตร์
	32	20	50	เขียนโปรแกรม

Advance Pointer

Linked List abstract data type (ADT)

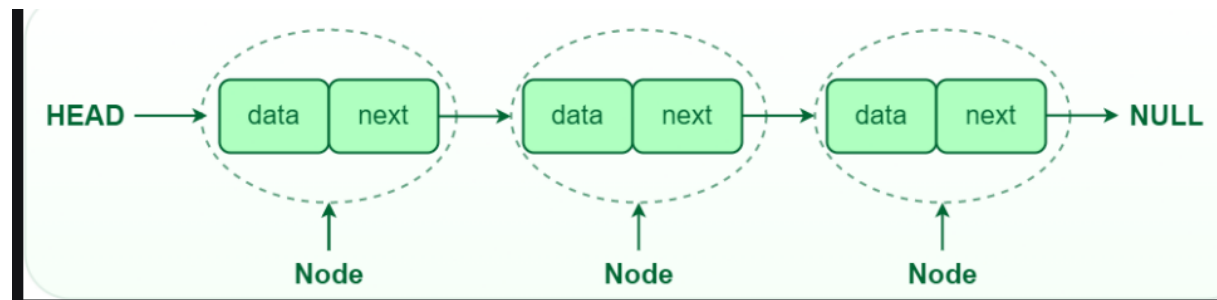


Suppose the data items on the list are ordered by ascending

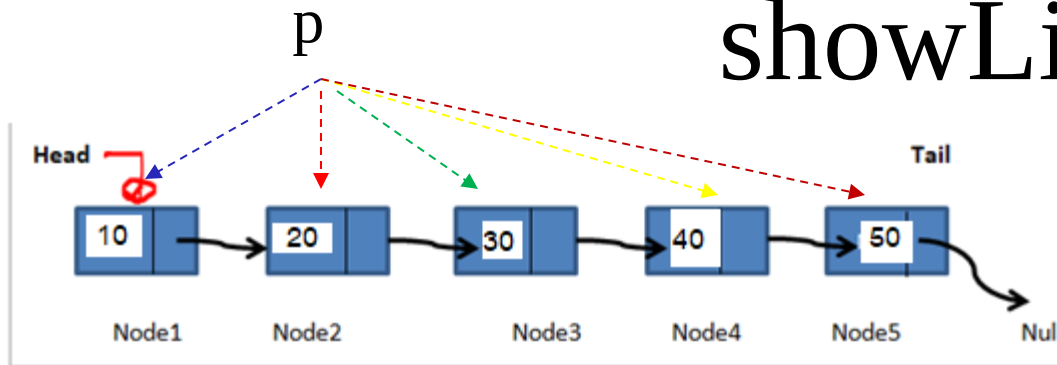
Operations :

1. Add
2. Find
3. Remove
4. Show

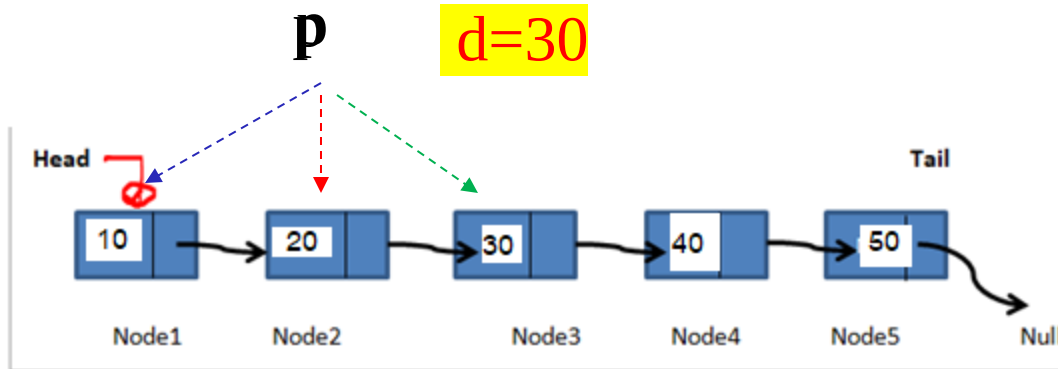
```
struct node {  
    int d;  
    struct node *next  
}
```



showList()



```
53 void showList()
54 {
55     int i = 0;
56     struct node *n = head;
57     while (n != NULL) {
58         cout << n->d << ",";
59         n = n->next;
60         i++;
61     }
62
63     cout << "\n number of nodes : " << i;
64
65 }
```



Find(int d)

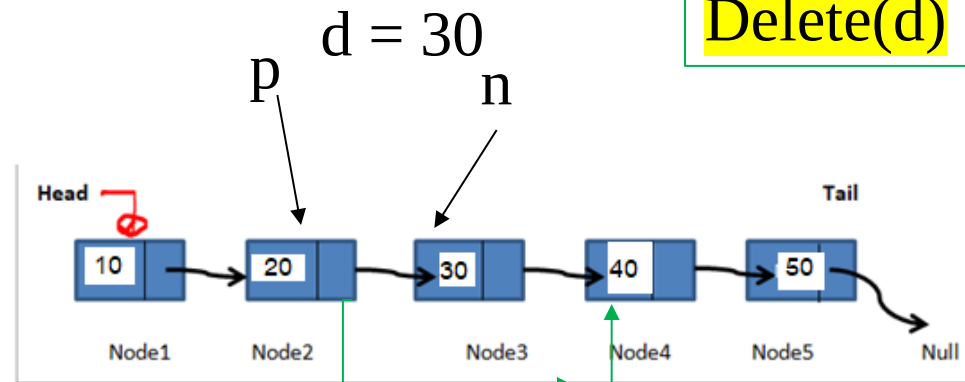
```

40  int Find(int d) {
41      int count = 0;
42      struct node *p = head;
43      while (p != NULL) {
44          count++;
45          if (p->d == d) {
46              return count;
47          }
48          p = p->next;
49      }
50      return false;
51  }

```

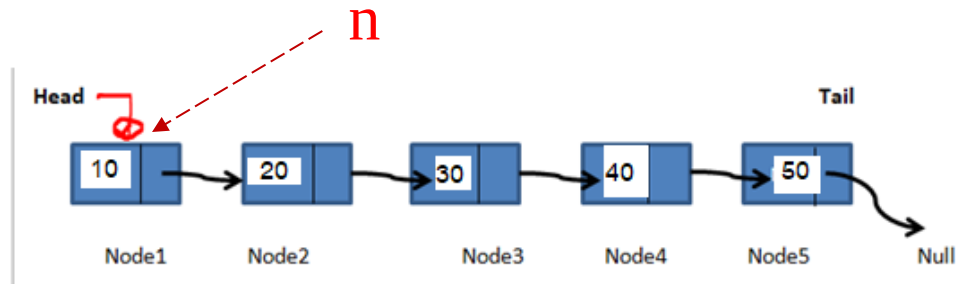
Delete(d)

```
67 bool Delete(int d) {
68     struct node *n = head;
69     struct node *p = NULL;
70     while (n != NULL) {
71         if (n->d == d) {
72             break;
73         }
74         p = n;
75         n = n->next;
76     }
77     if (n == NULL) {
78         return false;
79     }
80     //remove head
81     if (n == head) {
82         head = head->next;
83         delete n;
84     }
85     else {
86         p->next = n->next;
87         delete n;
88     }
89     return true;
90 }
91 }
```



d = 10

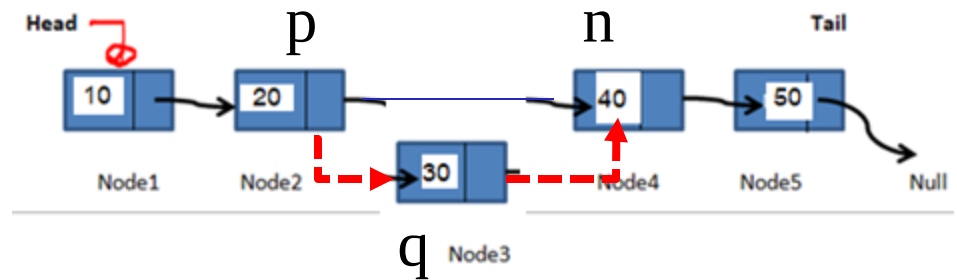
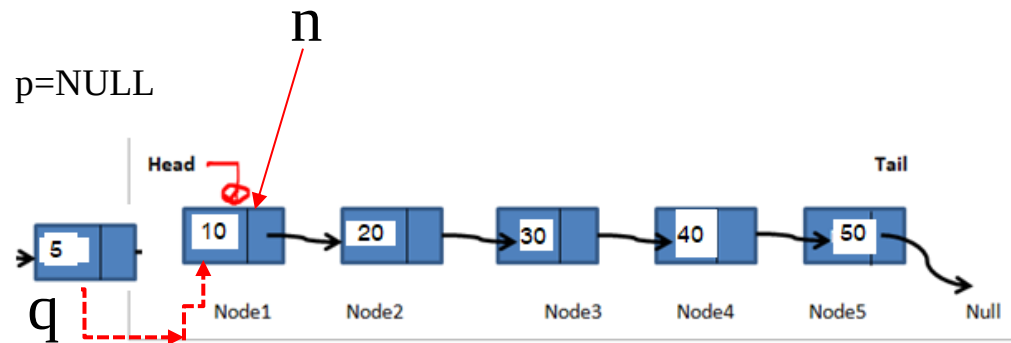
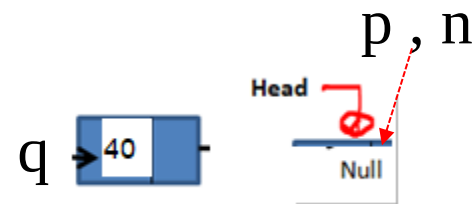
p = NULL



```

831 void Add(int d) {
832     struct node *q = new struct node;
833     q->d = d;
834     q->next = NULL;
835     struct node *n = head;
836     struct node *p = NULL;
837     while (n != NULL) {
838         if (n->d > d) {
839             break;
840         }
841         p = n;
842         n = n->next;
843     }
844
845     //first element
846     if (n == head) {
847         q->next = head;
848         head = q;
849         return;
850     }
851
852     p->next = q;
853     q->next = n;
854 }

```



Advance Memory : Linked List

สมมุติว่าข้อมูล อาจารย์คณะวิทยาศาสตร์มีดังนี้

Unknown number of data

เพศ (sex) : ชาย = 1 , หญิง = 2

อายุ (age) : ? ปี

สถานภาพสมรส (status) : โสด = 1 , แต่งงาน = 2 , อื่นๆ = 2

ปริญญาสูง (degree) : ตรี = 1 , โท = 2 , เอก = 3

จำนวนปีที่สอน (experience) : ? ปี

จงเขียน

1. โปรแกรมรับข้อมูลของอาจารย์ จนกระทั่ง sex มีค่าเท่ากับ -1
2. หาจำนวนปีเฉลี่ยของ อายุ และ จำนวนปีที่สอน

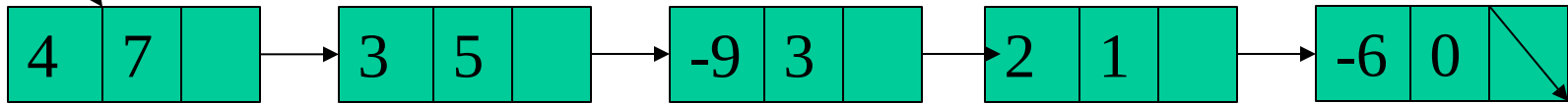
AVERAGE AGE = xx.xx Years

AVERAGE Experience = XX.XX Years

Ex1: จงเขียนโปรแกรมเก็บสมการพหุนาม และคำนวณค่า

head

เช่น $Y(X) = 4X^7 + 3X^5 - 9X^3 + 2X - 6$



โหนด	coef ค่าสัมประสิทธิ์	power ค่ายกกำลัง
$+4X^7$	+4	7
$+3X^5$	+3	5
$-9X^3$	-9	3
$+2X$	+2	1
-6	-6	0 โดย $X^0 = 1$

ถ้า ค่ายกกำลังมีค่าเท่ากับ 0 ให้หยุดรับข้อมูล โดยการป้อนข้อมูลให้ผู้ใช้ป้อนข้อมูลเรียงตามลำดับค่าของเลขยกกำลังสูง

ตัวอย่างการรับข้อมูล

ถ้า ค่ายกกำลังมีค่าเท่ากับ 0 ให้หยุดรับข้อมูล

0. add coef
1. calculate X
2. exit

ปรากฏดังรูป

```
coef=?4
power=?7
coef=?-9
power=?3
coef=?3
power=?5
coef=?-6
power=?0
coef=?2
power=?1
coef=?0
+4X^7+3X^5-9X^3+2X^1-6
```

ป้อนค่าสัมประสิทธิ์และค่ายกกำลัง

ป้อนค่าสัมประสิทธิ์เป็น 0
หยุดการรับข้อมูล

ผลการทำงานจะเรียงลำดับโหนด
ของข้อมูลตามกำลังจากมากไปน้อย

Advance Memory

- Pointer to Pointer
- Memory reallocation

Pointers to Pointers (C/C++)

- Pointers are also variables (storing addresses), so they have a memory location, so they also have an address
- Pointer to pointer – stores the address of a pointer variable

```
int x = 10, *p, **q;
```

```
p = &x ;
```

```
q = &p ;
```

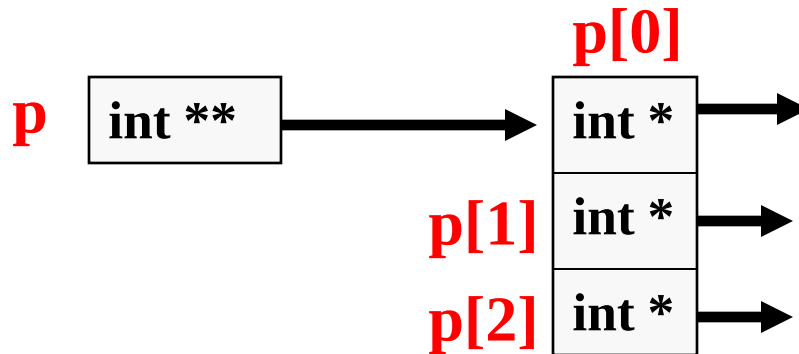
```
printf( "%d %d %d", x, *p, *(*q) );
```

```
will print 10 10 10 (since *q = p)
```

Allocating Pointer to Pointer (C/C++)

```
int **p;
```

```
p = (int **) malloc( 3 * sizeof(int *) );
```



realloc Example

```
float *nums;
int I;

nums = (float *) calloc(5, sizeof(float));
/* nums is an array of 5 floating point values */

for (I = 0; I < 5; I++)
    nums[I] = 2.0 * I;
/* nums[0]=0.0, nums[1]=2.0, nums[2]=4.0, etc. */

nums = (float *) realloc(nums, 10 * sizeof(float));
/* An array of 10 floating point values is allocated,
   the first 5 floats from the old nums are copied as
   the first 5 floats of the new nums, then the old
   nums is released */
```